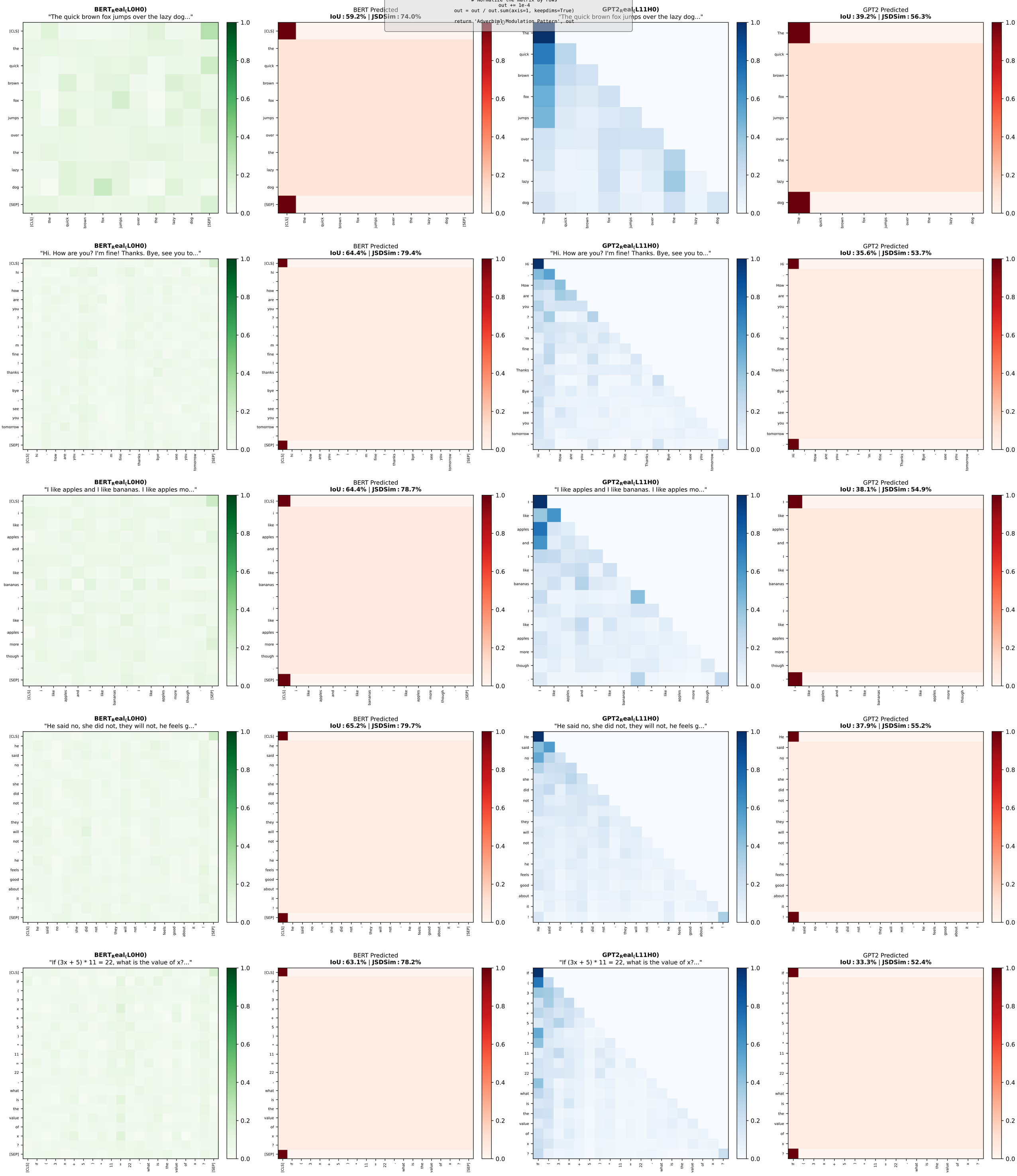
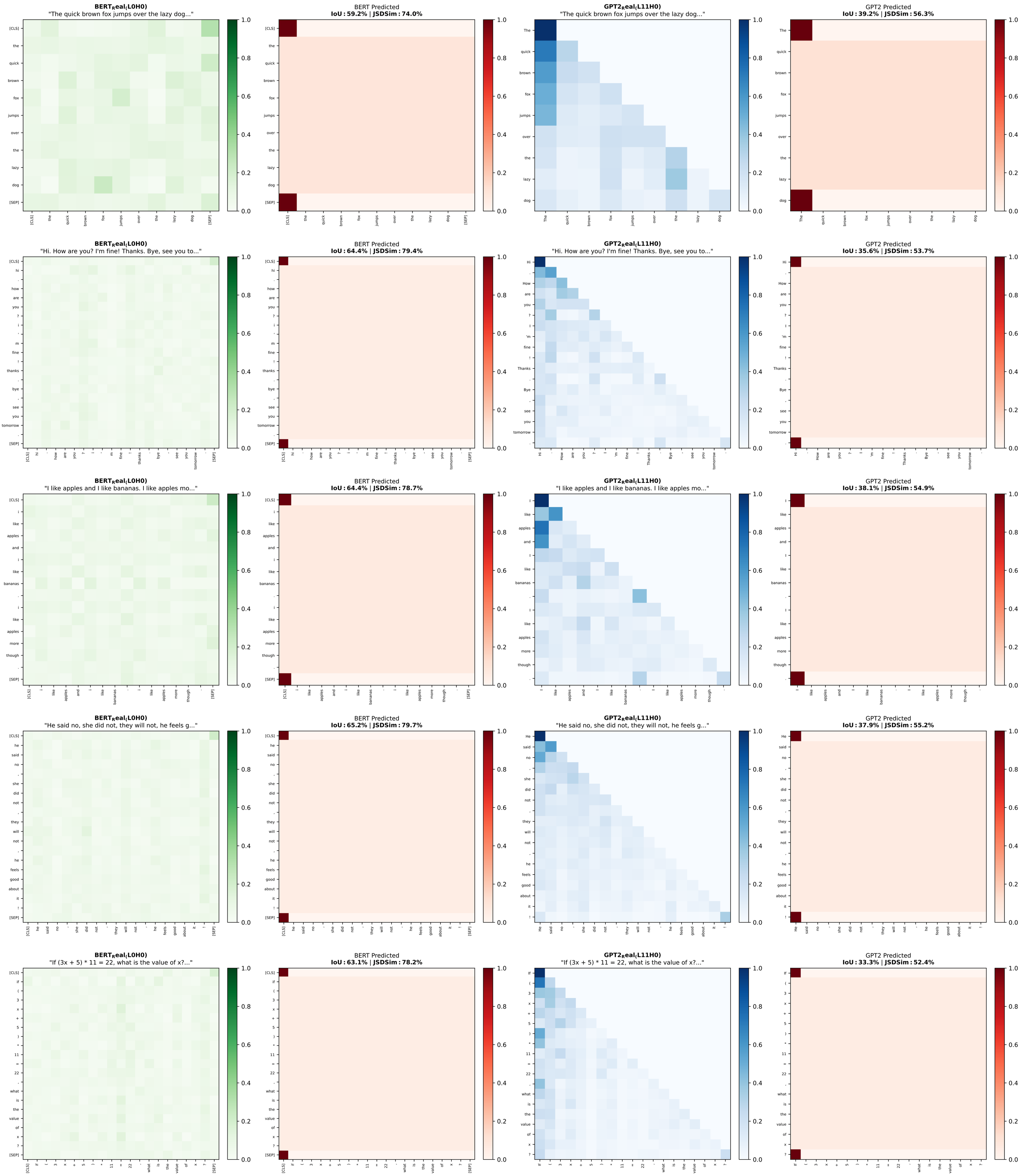
Pattern: adverbial_modulation

```
def adverbial_modulation(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors='pt')
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))

                      # Process sentence with spaCy to align tokens
                             words = sentence.split()
                            doc = nlp(' '.join(words))

             # Create a mapping to handle tokenization alignment differences
                             spaCy_to_tokenizer = {}
                               tokenizer_index = 1
                                for word in doc:
                     for _ in range(len(tokenizer.tokenize(word.text))):
                          spaCy_to_tokenizer[word.i] = tokenizer_index
                                  tokenizer_index += 1

        # Check for adverbs and link them to their governing verbs, if applicable
                                 for token in doc:
                                if token.pos_ == 'ADV':
                           adverb_index = spaCy_to_tokenizer.get(token.i)
                              for child in token.head.children:
                       if child.dep_ in {'advcl', 'conj', 'xcomp', 'adjunct'}:
                              head_index = spaCy_to_tokenizer.get(child.i)
                                 if adverb_index and head_index:
                            out[adverb_index, head_index] = 1  # Direct attention
                           out[head_index, adverb_index] = 1  # Ensure bidirectional

                           # Ensure [CLS] and [SEP] are attended
                                        out[0, 0] = 1
                                       out[-1, 0] = 1

                               # Normalize the matrix by rows
                                         out += 1e-4
                        out = out / out.sum(axis=1, keepdims=True)
                        return 'Adverbial Modulation Pattern', out
```



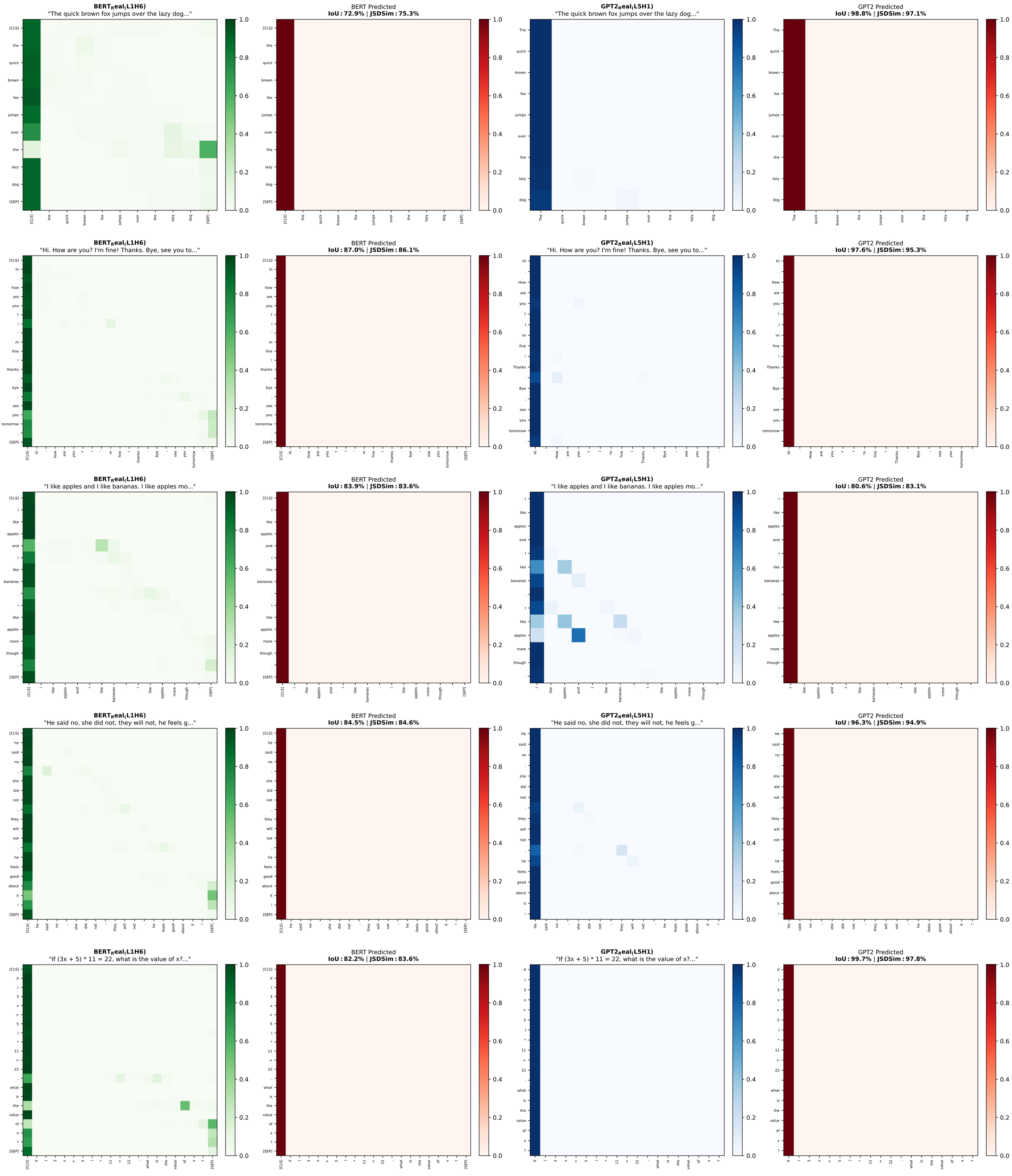
Pattern: appositive_phrase_attention

```
def appositive_phrase_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
    # Tokenize the sentence and identify appositive punctuation marks like ',' or '()'
                             tokens = sentence.split()
                # Assuming tokens and spaCy entities have been matched
            comma_indices = [i for i, tok in enumerate(tokens) if tok == ',']

    # If there are appositive phrases, link the head of the sentence part around commas
                             if len(comma_indices) > 1:
                           for i in range(len(comma_indices) - 1):
                                   start = comma_indices[i]
                                   end = comma_indices[i+1]
                        # Make those segments attend strongly to themselves
                               out[start+1:end+1, start+1:end+1] = 1

                        # Ensure [CLS] and [SEP] have some attention
                                    out[0, 0] = 1
                                   out[-1, 0] = 1

                            # Normalize the attention matrix
                     out += 1e-4  # Add some smoothing to avoid pure zeros
                          out = out / out.sum(axis=1, keepdims=True)

                           return "Appositive Phrase Attention", out
```



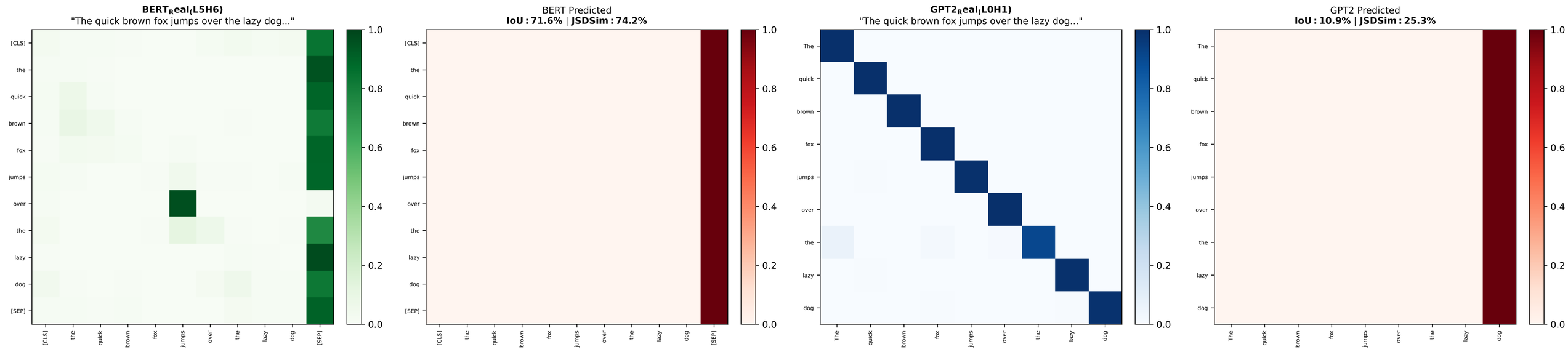
Pattern: cls_attention

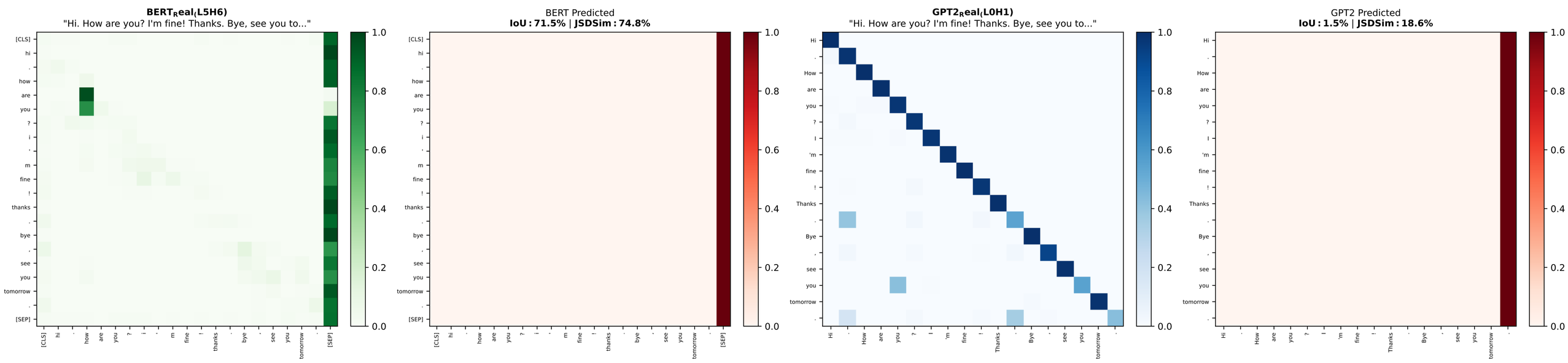
```
def cls_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    out[:, 0] = 1
    return "CLS Pattern", out
```

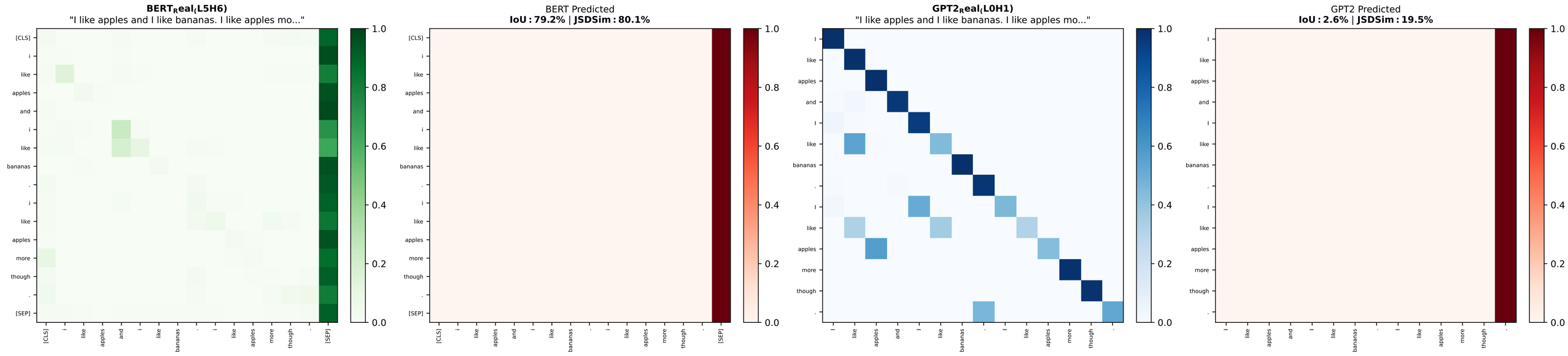


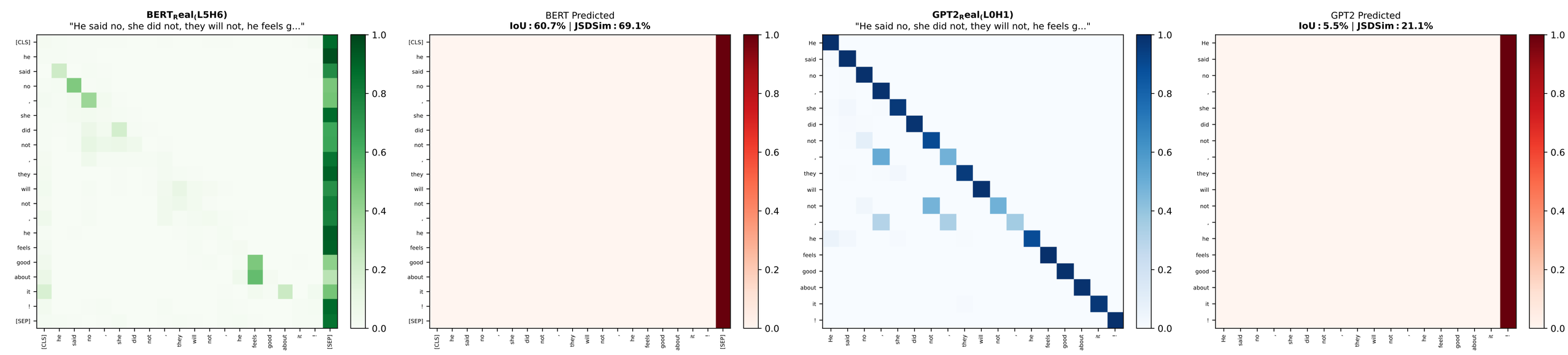
Pattern: complement_adjunct_relationship

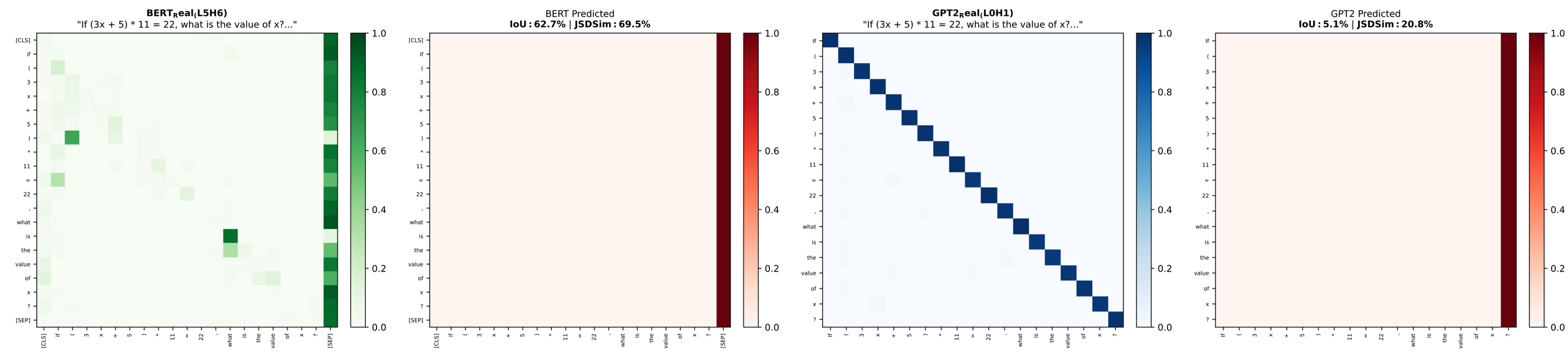
```
def complement_adjunct_relationship(sentence: str, tokenizer: PreTrainedTokenizerBase) -> "Tuple[str, np.ndarray]":
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Define hardcoded relationships based on observed patterns
    # These are generalized rules observed across the sample data
    complement_pairs = [['to', 'play'], ['to', 'difficult'], ['with', 'play'],
                        ['with', 'share'], ['for', 'difficult']]

    def find_pairs(splitted_sentence, pair):
        indices_1 = [i for i, word in enumerate(splitted_sentence) if word == pair[0]]
        indices_2 = [i for i, word in enumerate(splitted_sentence) if word == pair[1]]
        return indices_1, indices_2

    words = sentence.lower().split()

    for pair in complement_pairs:
        indices_1, indices_2 = find_pairs(words, pair)
        for i1 in indices_1:
            for i2 in indices_2:
                out[i1+1, i2+1] = 1  # Adjust for CLS token
                out[i2+1, i1+1] = 1

    # Ensure every token has an attention weight
    for row in range(len_seq):
        if out[row].sum() == 0:
            out[row, -1] = 1.0

    return "Complement/Adjunct Relationship", out
```





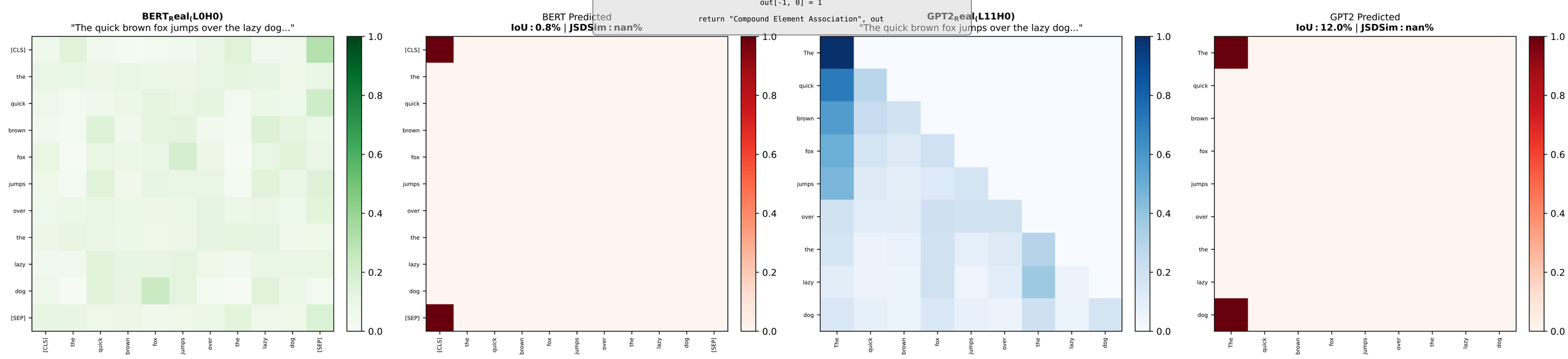


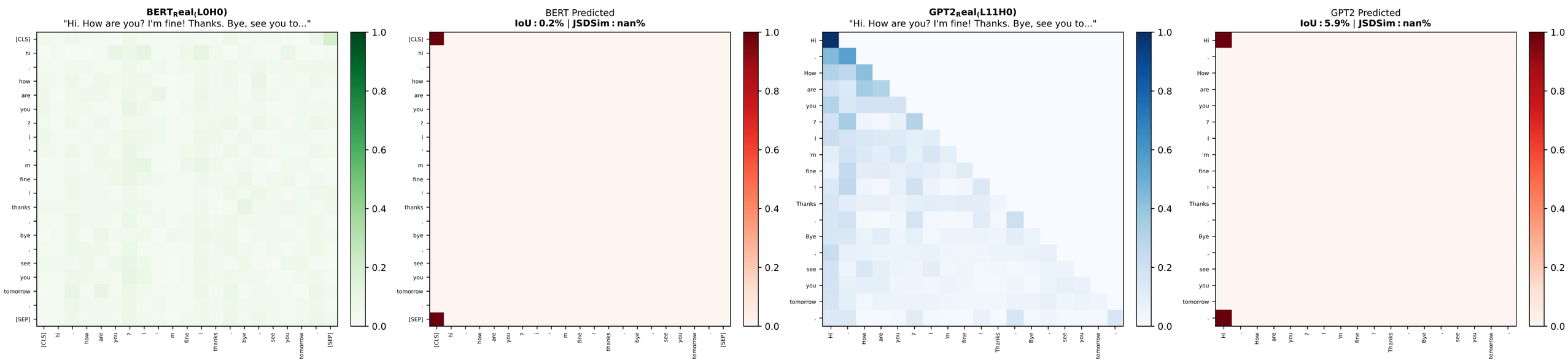


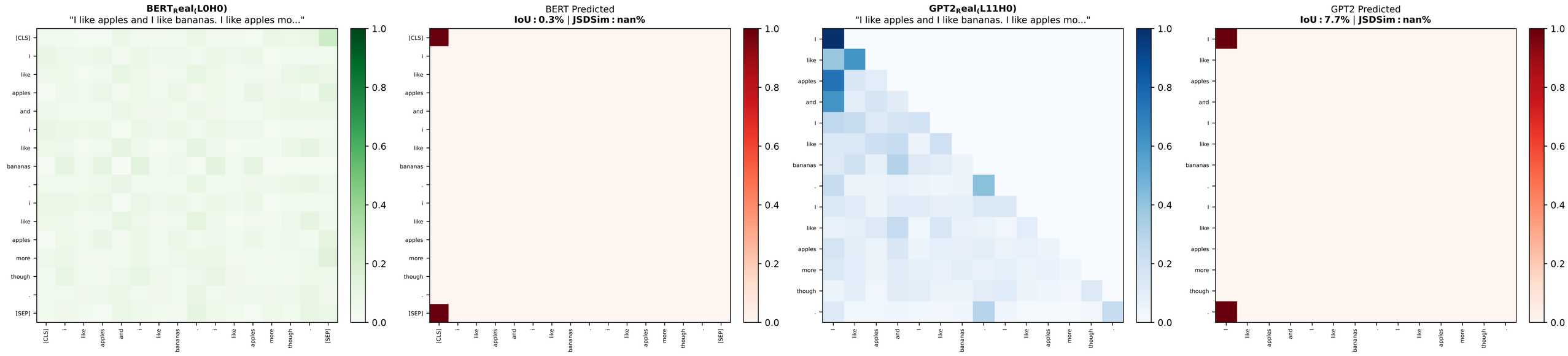


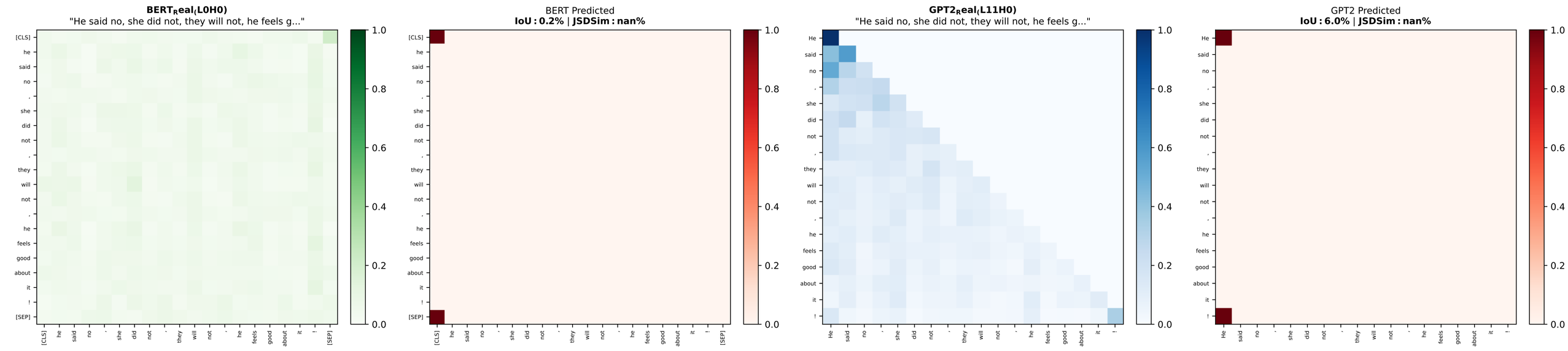
Pattern: compound_element_association

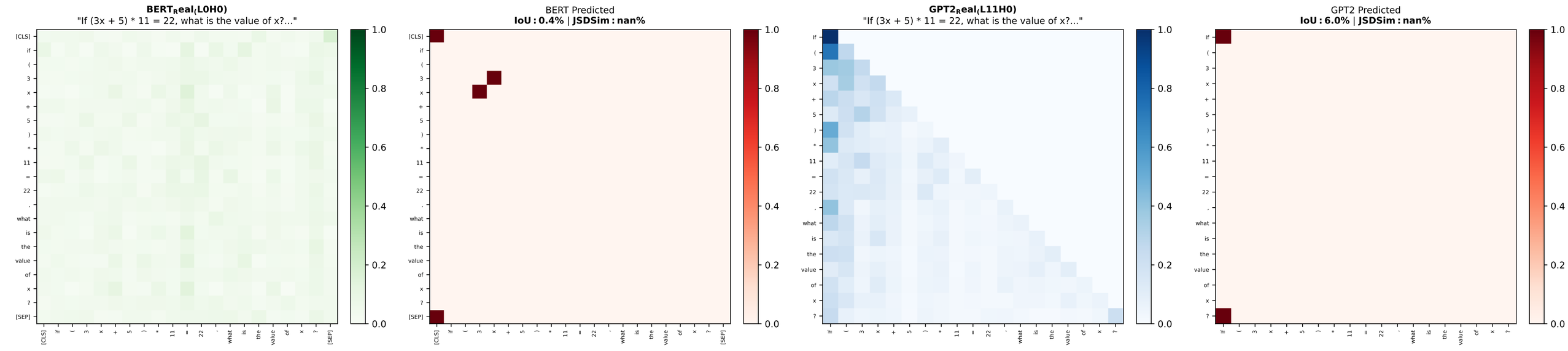
```
def compound_element_association(sentence: str, tokenizer: PreTrainedTokenizerBase):
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))

            # Split sentence into words assuming tokenizer gives word_ids
                     word_tokens = toks.word_ids(batch_index=0)
                                 word_to_tokens = {}

                    # Map each word to its corresponding tokens
                    for idx, word_id in enumerate(word_tokens):
                                  if word_id is None:
                                         continue
                          if word_id not in word_to_tokens:
                               word_to_tokens[word_id] = []
                        word_to_tokens[word_id].append(idx)

                # Identifying compound elements within the tokens
            # Look for pattern where subparts are related within compounds
                for word_id, token_indices in word_to_tokens.items():
                if len(token_indices) > 1:  # Indicates a compound element
                                for token_i in token_indices:
                                  for token_j in token_indices:
                                        if token_i != token_j:
                                         out[token_i, token_j] = 1

                                # Normalize attention
                         row_sums = out.sum(axis=1, keepdims=True)
     np.divide(out, row_sums, out=np.zeros_like(out, dtype=float), where=row_sums!=0)

                          # Including [CLS] and [SEP] importance
                                        out[0, 0] = 1
                                       out[-1, 0] = 1

                        return "Compound Element Association", out
```





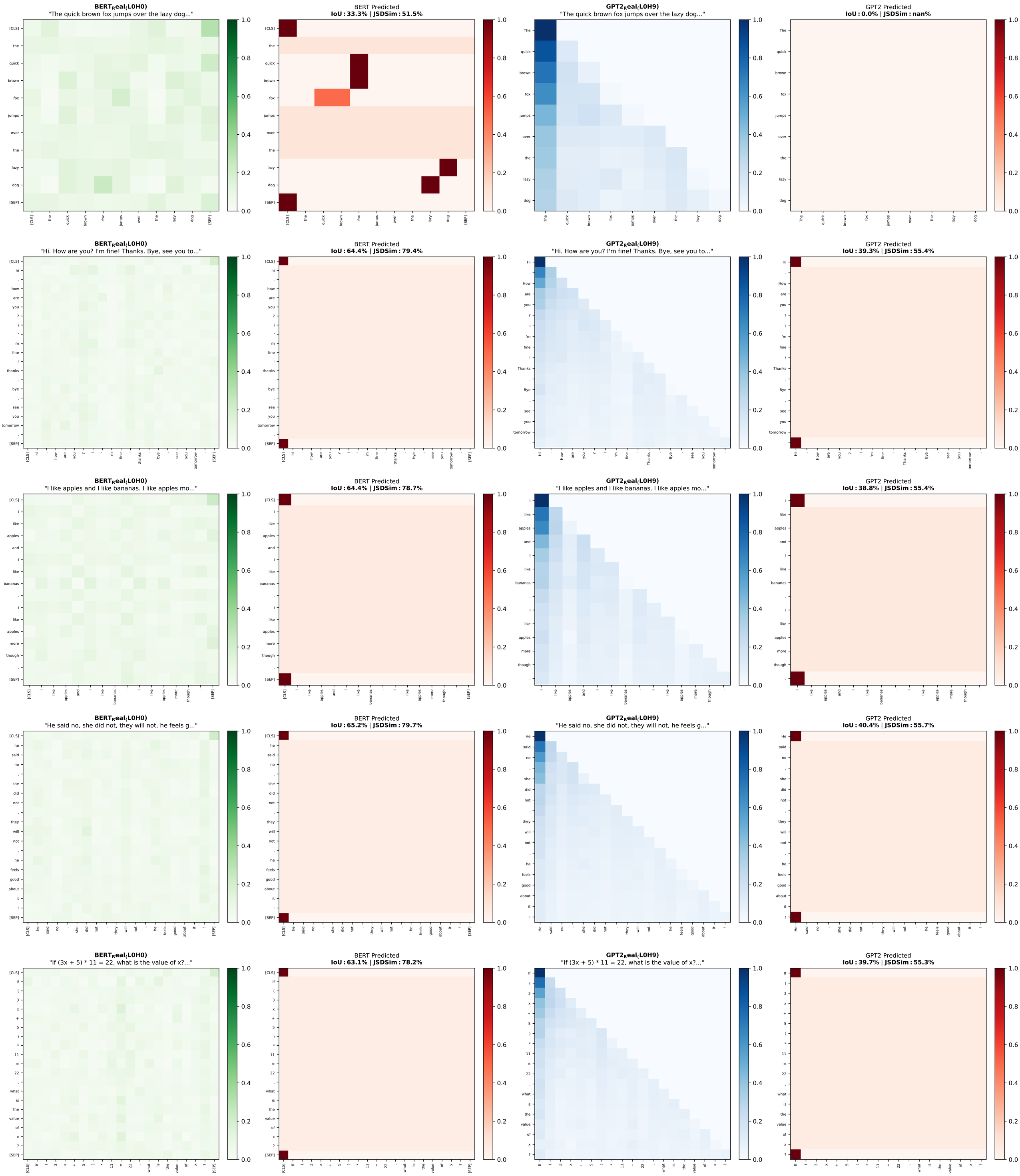






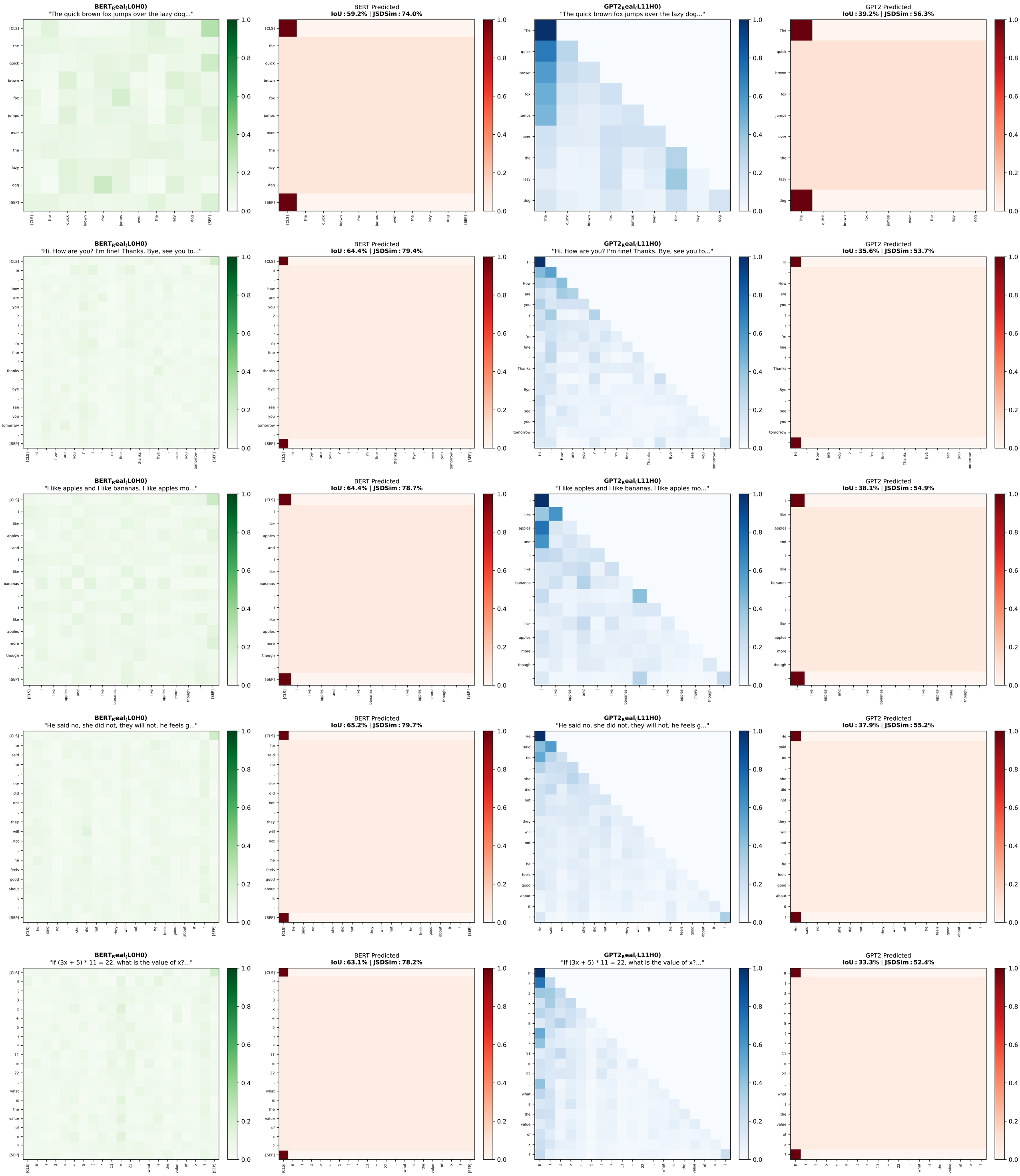
Pattern: compound_modifier_attention

```
def compound_modifier_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    nlp = spacy.load("en_core_web_sm")
    doc = nlp(sentence)
token_map = {token.i: idx for idx, token in enumerate(doc)}

    for token in doc:
if token.dep_ in {"amod", "compound"}:  # Modifier relationships
    head_idx = token_map.get(token.head.i, -1)
    if 1 <= head_idx < len_seq:
        token_idx = token_map.get(token.i, -1)
out[token_idx + 1, head_idx + 1] = 1  # Applying the modifier-to-head attention
    out[head_idx + 1, token_idx + 1] = 1  # Symmetrically

    out[0, 0] = 1  # CLS attends to CLS
    out[-1, 0] = 1  # SEP attends to CLS

# Normalize the attention pattern across each token's row
out += 1e-4  # Adding small value for numerical stability
    out = out / out.sum(axis=1, keepdims=True)
return "Compound Formation - Modifier Head Attention", out
```



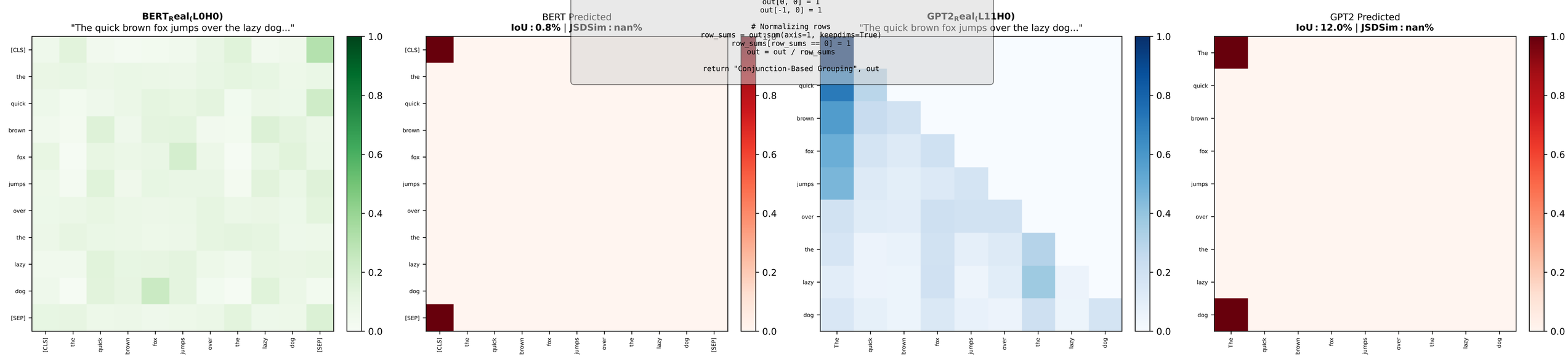
Pattern: compound_word_attention_pattern

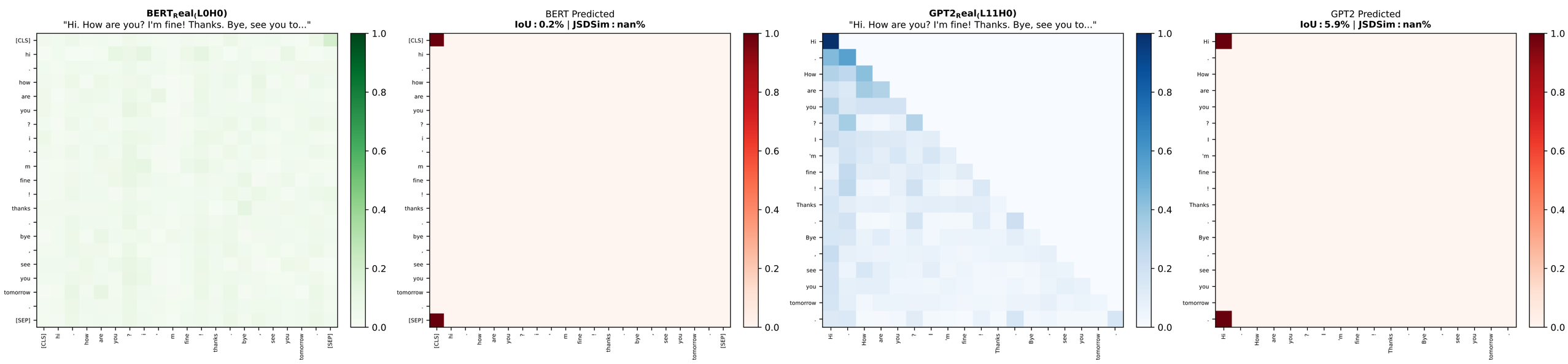
```
def compound_word_attention_pattern(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors='pt')
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
                                words = sentence.split()

        # Simple heuristic: treat sequences with two or more hash symbols as compound words
                          for idx, token in enumerate(words):
                                        if '##' in token:
                      # Find the starting index of the compound part
                            compound_idx = idx - token.count('##')
                out[compound_idx, idx + 1] = 1  # shift by 1 to account for [CLS]
                              out[idx + 1, compound_idx] = 1

                           out[0, 0] = 1  # Attention to [CLS]
                          out[-1, 0] = 1  # Attention to [SEP]

                 # Normalize attention matrix to sum to 1 over each row
                 out += 1e-4  # tiny value to avoid zero division errors
                       out = out / out.sum(axis=1, keepdims=True)
                         return 'Compound Word Attention', out
```

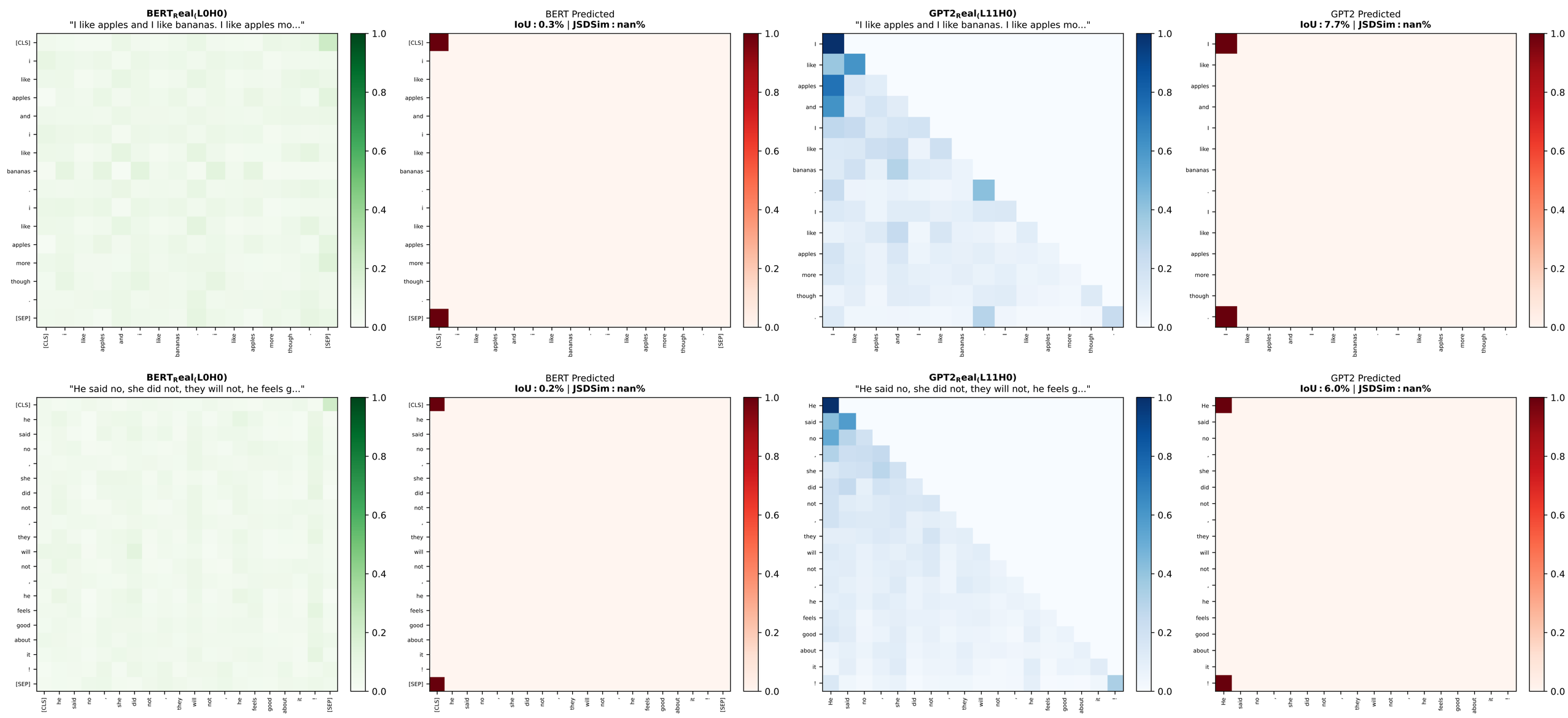


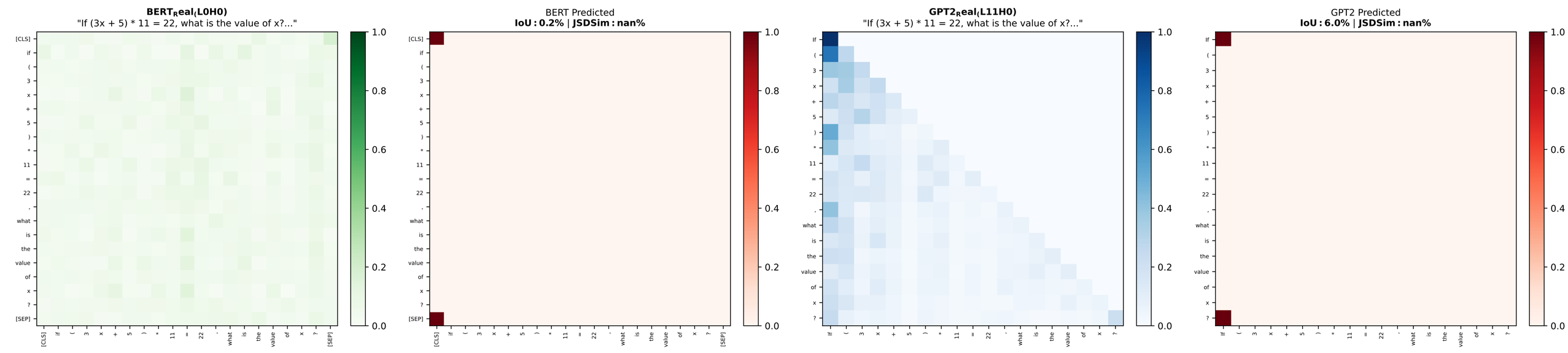
Pattern: conjunction_based_grouping

```
def conjunction_based_grouping(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    doc = nlp(sentence)
    word_ids = {token.idx: i for i, token in enumerate(doc)}

    # Identifying conjunctions and their respective grouped nouns/phrases
    conjunction_indices = [i for i, token in enumerate(doc) if token.pos_ == 'CCONJ']
    for conj_index in conjunction_indices:
        left_tree = doc[conj_index].lefts  # Tokens on the left of the conjunction
        right_tree = doc[conj_index].rights  # Tokens on the right of the conjunction

        left_indices = [word_ids[token.idx] for token in left_tree if token.idx in word_ids]
        right_indices = [word_ids[token.idx] for token in right_tree if token.idx in word_ids]

        # Create links within each group and across the conjunction
        for i in left_indices:
            for j in right_indices:
                out[i+1, j+1] = 1
                out[j+1, i+1] = 1
        for i in left_indices:
            for j in left_indices:
                if i != j:
                    out[i+1, j+1] = 1
        for i in right_indices:
            for j in right_indices:
                if i != j:
                    out[i+1, j+1] = 1

    # Ensure [CLS] attends to itself and [SEP] to [CLS]
    out[0, 0] = 1
    out[-1, 0] = 1

    # Normalizing rows
    row_sums = out.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1
    out = out / row_sums

    return "Conjunction-Based Grouping", out
```



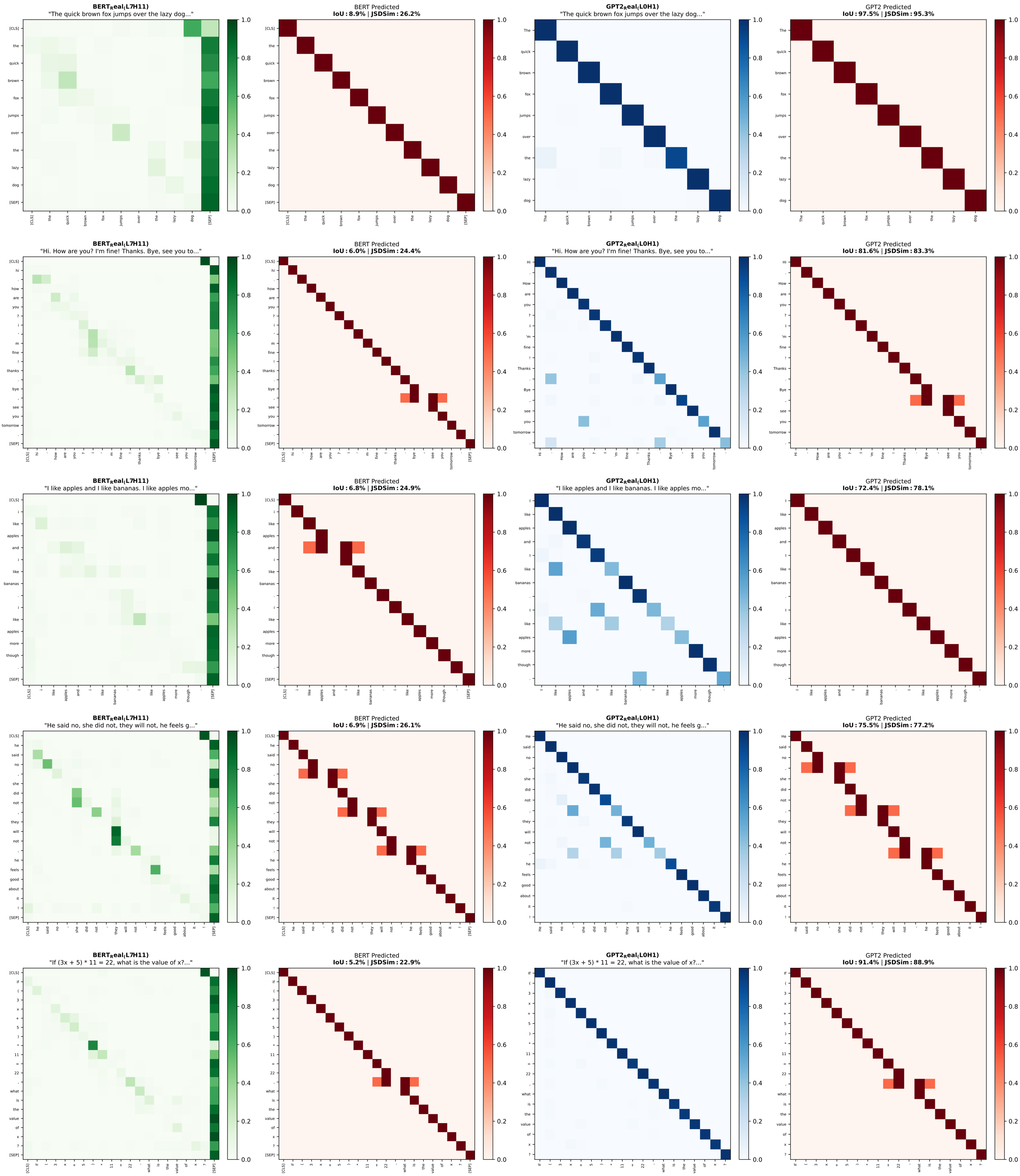






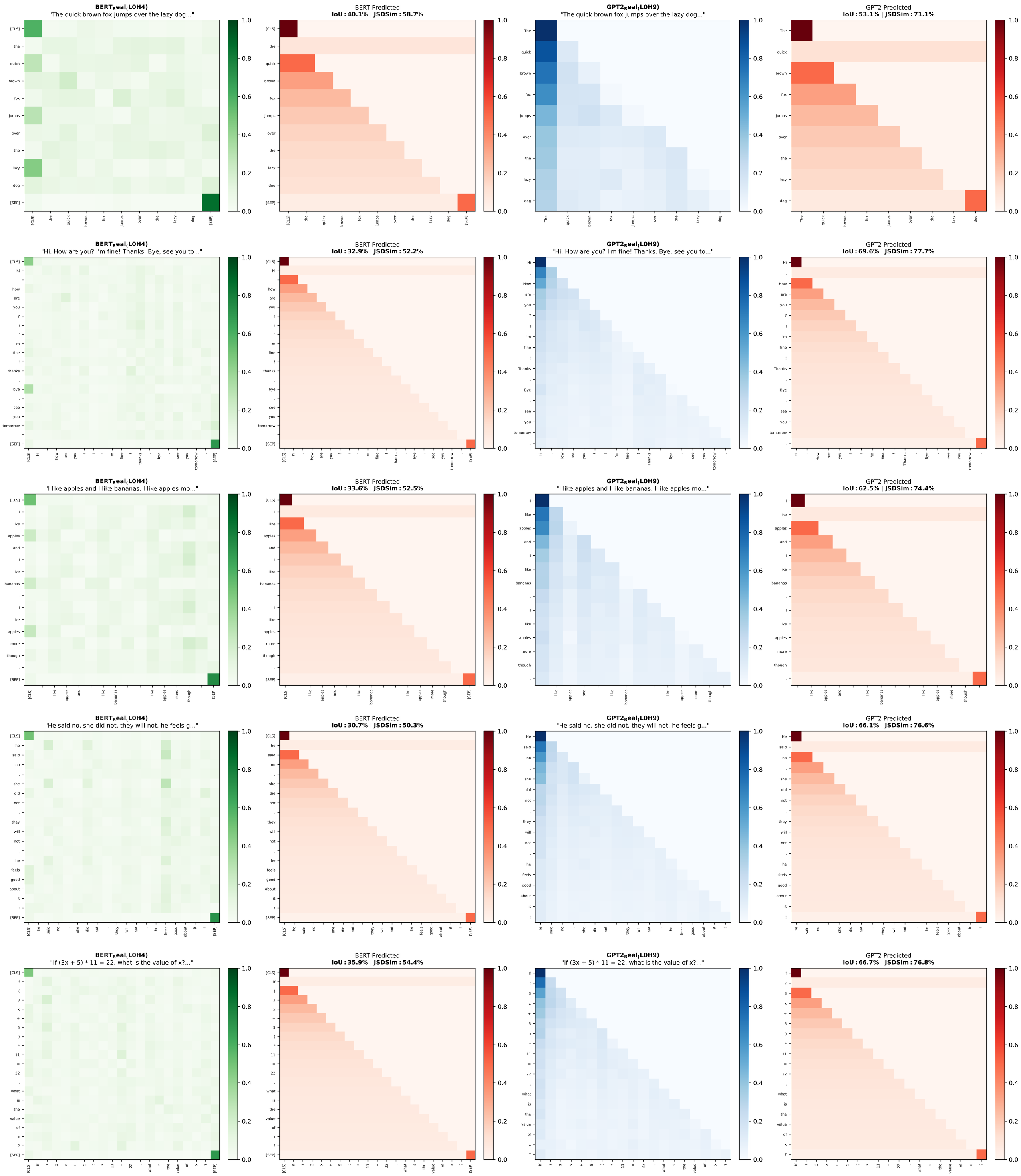
Pattern: conjunction_resolution

```
def conjunction_resolution(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    words = tokenizer.convert_ids_to_tokens(toks.input_ids[0])
    conjunctions = {"and", "or", "but", ","}

    conjunction_indices = [i for i, word in enumerate(words) if word.lower() in conjunctions]

    for idx in conjunction_indices:
        if idx > 0 and idx < len_seq - 1:
            # Attention to the previous and next word of the conjunction
            out[idx, idx - 1] = 1
            out[idx, idx + 1] = 1
            if idx > 1:
                # Have some backwards reference to earlier conjunctions or main verbs
                out[idx, idx - 2] = 0.5
            if idx < len_seq - 2:
                # Have some forward reference to subsequent conjunctions or key elements
                out[idx, idx + 2] = 0.5

    # Ensure no row is all zeros
    for row in range(len_seq):
        if out[row].sum() == 0:
            out[row, row] = 1.0

    return "Coordination and Conjunction Resolution", out
```



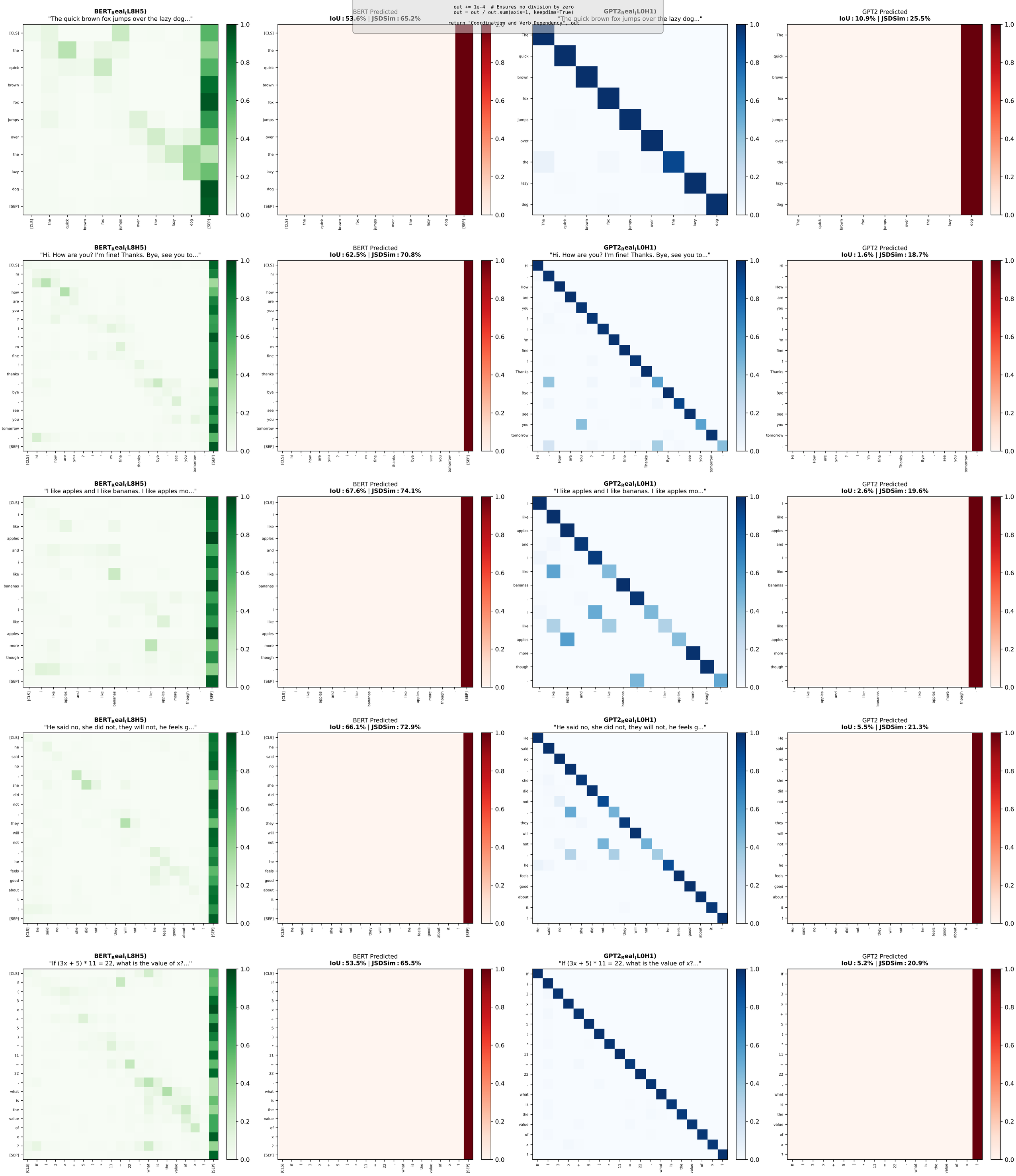
Pattern: contextual_anchoring

```
def contextual_anchoring(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    # Tokenize the input sentence
    toks = tokenizer([sentence], return_tensors='pt')
    len_seq = len(toks.input_ids[0])  # Get the length of the sequence
    out = np.zeros((len_seq, len_seq))  # Initialize the attention matrix

    # Loop over each token, with the first token having the highest self-attention
    for i in range(1, len_seq):
        if i == 1:
            # The first word receives the strongest anchoring attention
            out[i, :] = 1.0  # Anchoring to the first token (CLS-like behavior without dominance)
        else:
            # The rest of the sentence's words receive progressively less attention
            # but are still rooted in the initial segment
            out[i, :i] = 1.0 / (i)

    # Since special tokens usually have fixed heads, let them attend to themselves
    out[0, 0] = 1
    out[-1, -1] = 1

    # Normalize to mimic attention weights
    out /= out.sum(axis=1, keepdims=True)

    # Return the identified pattern name and the built attention matrix
    return 'Sentence-Initiated Contextual Anchoring', out
```



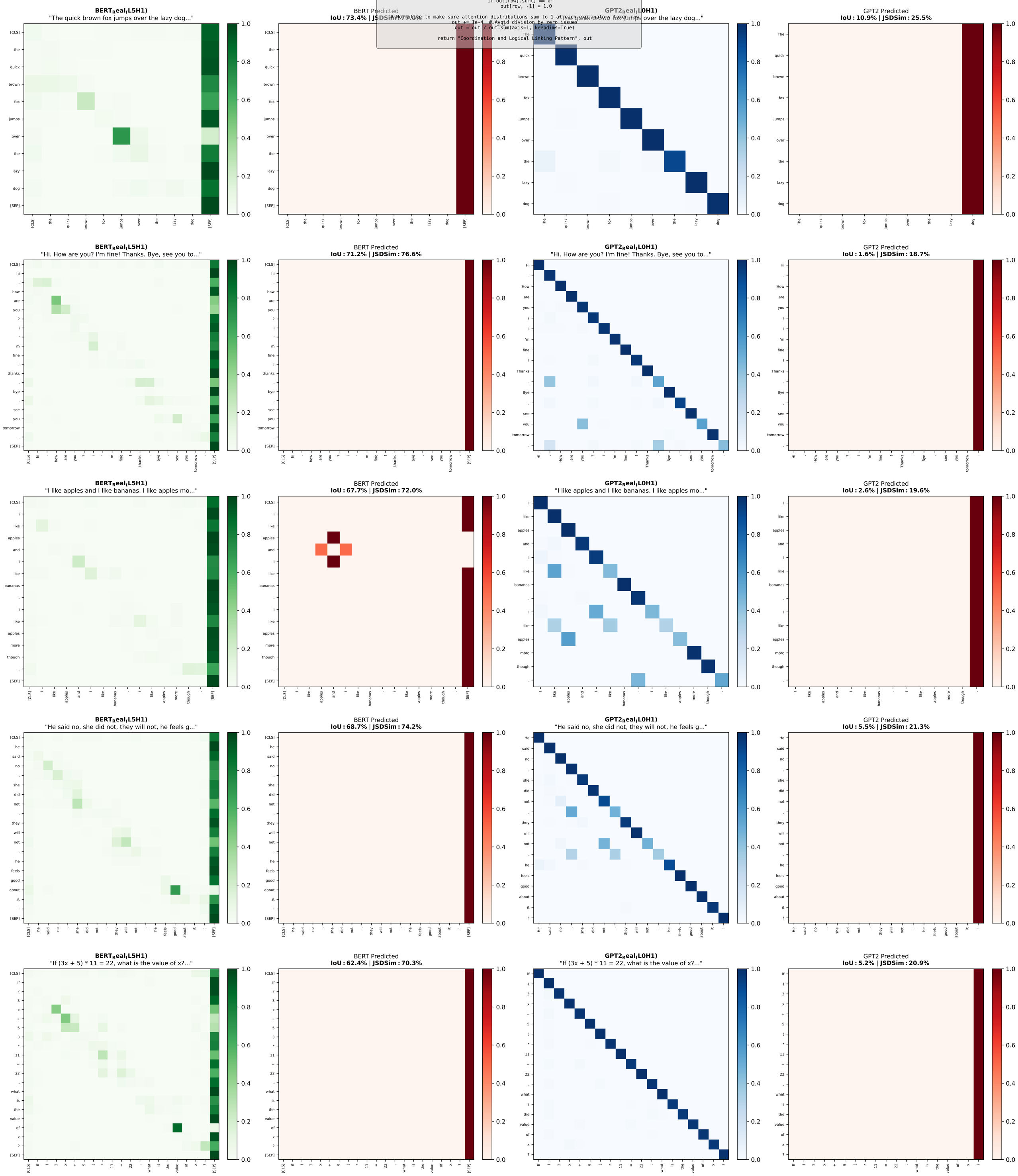
Pattern: coord_and_verb_dependency

```
def coord_and_verb_dependency(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                    toks = tokenizer([sentence], return_tensors="pt")
                                            len_seq = len(toks.input_ids[0])
                                           out = np.zeros((len_seq, len_seq))

                                          # Define indices for special tokens
                                                     cls_idx = 0
                                                 sep_idx = len_seq - 1

            # These hypothetical rules aim to capture verbs and their related coordination compounds
                  # Note: The indices are hypothetical and align with overall style convention
                     # Use dictionary to map token indices to words for indexed operations
                         tok_map = {i: tok for i, tok in enumerate(toks.input_ids[0])}

                           # Simplified interpretation of observed attention patterns:
                            # Key pattern: and|verb and between coordinated structures
                           tokens = tokenizer.convert_ids_to_tokens(toks.input_ids[0])
                   verbs = [i for i, token in enumerate(tokens) if token.startswith('##')]
             coord_conjs = [i for i, token in enumerate(tokens) if token in ['and', 'but', 'or']]

                                  # Examples of coordination triggering
                                        for coord in coord_conjs:
                  # Assume verbs tend to 'command' coordination especially when 'and' is present
                                              for verb in verbs:
                         if verb < coord < sep_idx:  # hypothetical logic for selective tuning
                                                  out[coord, verb] = 1
                                                  out[verb, coord] = 1

                            # Handling basic normalization assuming no row is zero
                                        for row in range(len_seq):
                                           if out[row].sum() == 0:
                                             out[row, sep_idx] = 1.0

                                   out += 1e-4  # Ensures no division by zero
                                   out = out / out.sum(axis=1, keepdims=True)

                                 return "Coordination and Verb Dependency", out
```



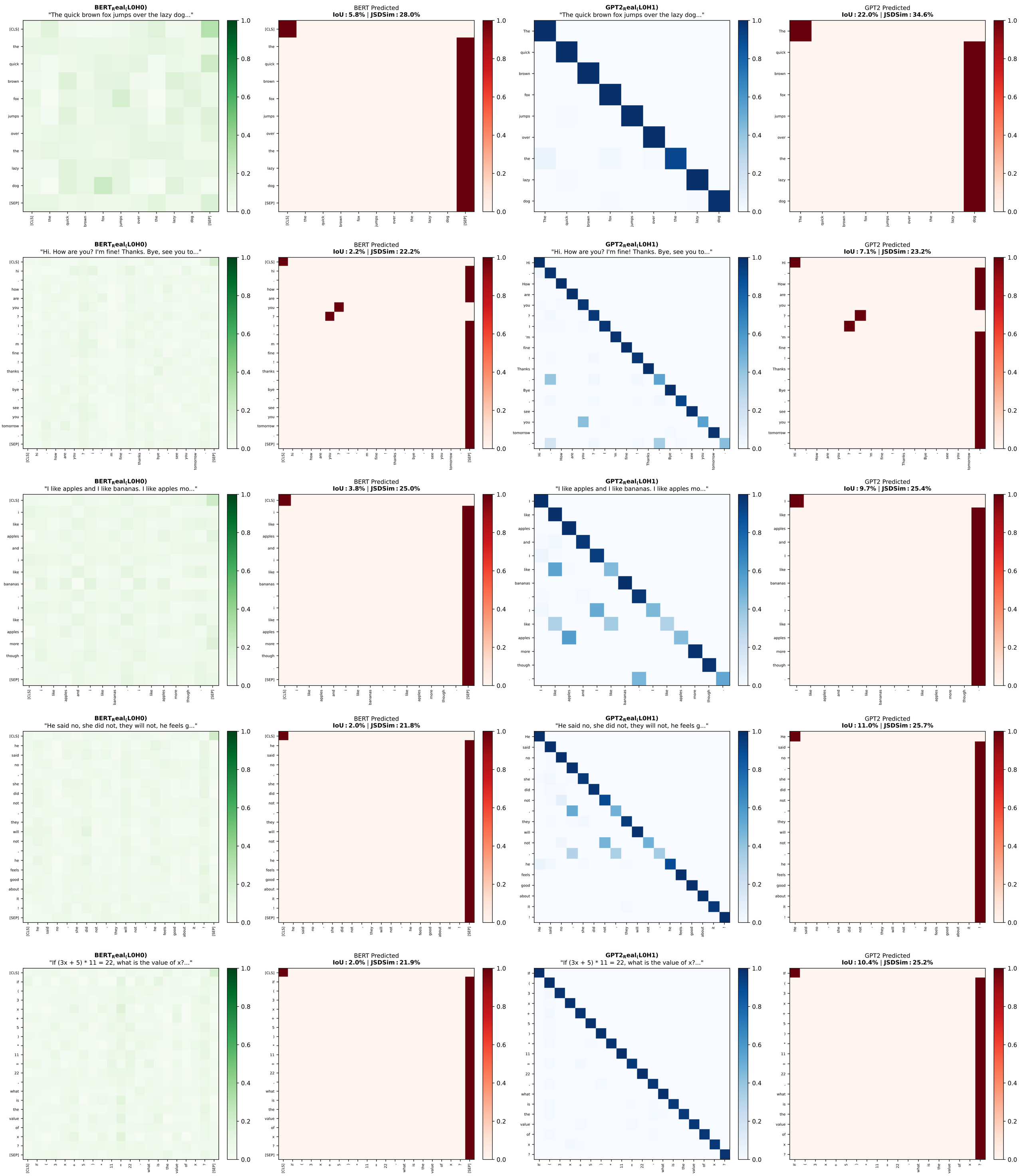
Pattern: coordination_attention

```
def coordination_attention(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors='pt')
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Tokens that typically involve coordination conjunctions or logical connectors
    coordination_tokens = {"and", "with", "for"}

    # Decoding back to words to compare
    token_words = tokenizer.convert_ids_to_tokens(toks.input_ids[0])

    # Build a coordination mapping
    coordination_indices = []
    for idx, token in enumerate(token_words):
        if token in coordination_tokens:
            coordination_indices.append(idx)

    # Apply attention pattern
    for idx in coordination_indices:
        # Apply strong attention from the coordination word to its immediate neighbors
        if idx > 0:
            out[idx, idx-1] = 1.0  # Previous token
        if idx < len_seq - 1:
            out[idx, idx+1] = 1.0  # Next token
        # Reverse attention to coordination word itself for emphasis from linked tokens
        if idx > 0:
            out[idx-1, idx] = 1.0
        if idx < len_seq - 1:
            out[idx+1, idx] = 1.0

    # Ensure no row in out is all zeros
    for row in range(len_seq):
        if out[row].sum() == 0:
            out[row, -1] = 1.0

    # Normalize to make sure attention distributions sum to 1 at each explanatory token row
    out += 1e-4  # Avoid division by zero issues
    out = out / out.sum(axis=1, keepdims=True)

    return "Coordination and Logical Linking Pattern", out
```



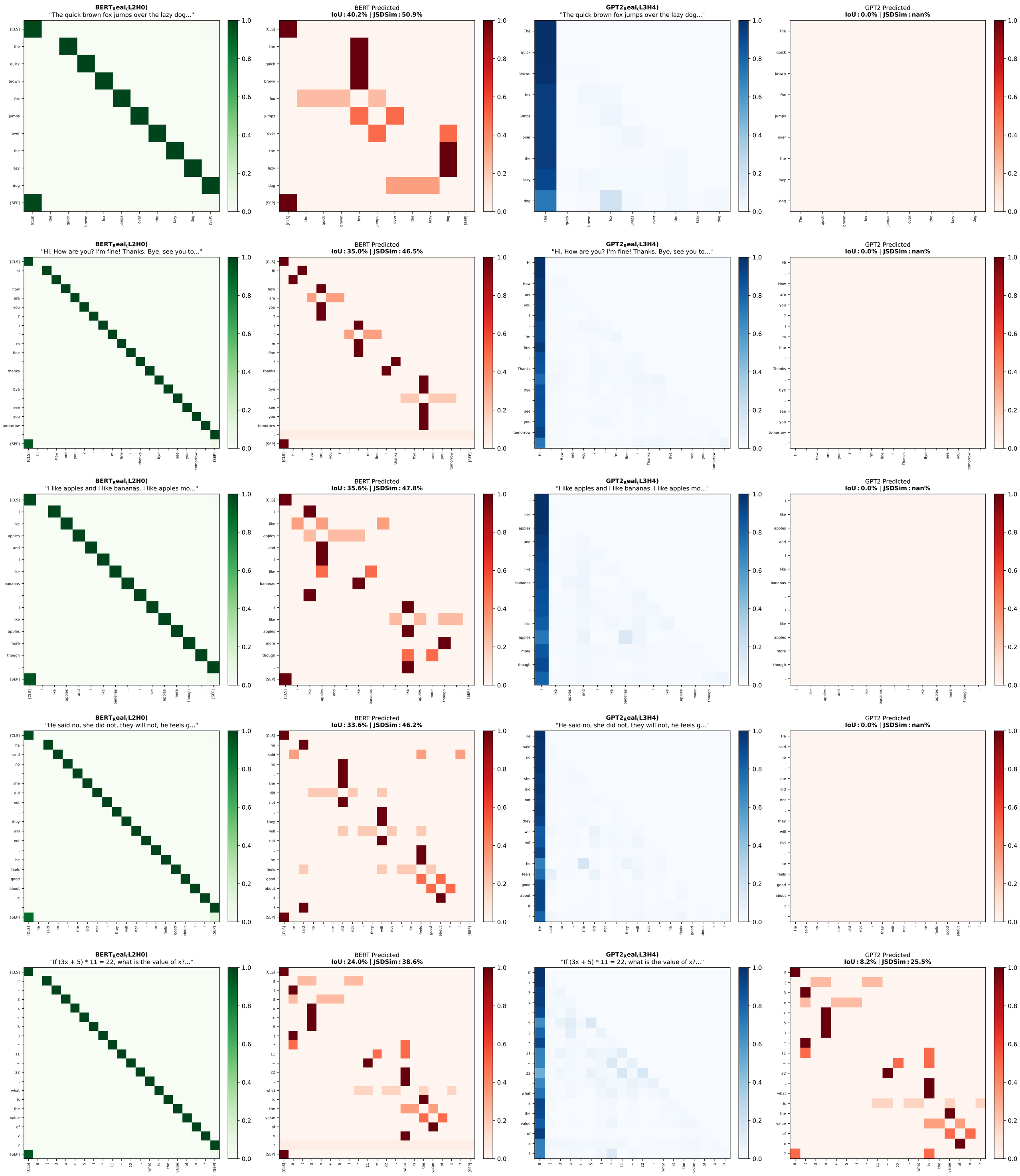
Pattern: coreference_resolution

```
def coreference_resolution(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                        len_seq = len(toks.input_ids[0])
                                       out = np.zeros((len_seq, len_seq))

                                       # Use spaCy to parse the sentence
                                                doc = nlp(sentence)
            token_to_index = {tok.idx: i+1 for i, tok in enumerate(doc) if tok.text.strip()}

                         # Iterate over tokens and activate coreference patterns
                                  for i, token in enumerate(doc):
                              if token.pos_ == "PRON" or token.dep_ == "nsubj":
                              # Find previous tokens possibly acting as antecedents
                                                for j in range(i):
                                         if doc[j].pos_ in ["NOUN", "PROPN"]:
                                      if j in token_to_index and i in token_to_index:
                                         out[token_to_index[j], token_to_index[i]] = 1
                                         out[token_to_index[i], token_to_index[j]] = 1

      # Assign self attention for cls ([0, 0]) and eos ([-1, -1]), and default attention to [eos, ...]
                                                 out[0, 0] = 1
                                                out[-1, -1] = 1
                                         for row in range(len_seq):
                                             if out[row].sum() == 0:
                                                 out[row, -1] = 1.0

                                 return "Coreference Resolution Pattern", out
```



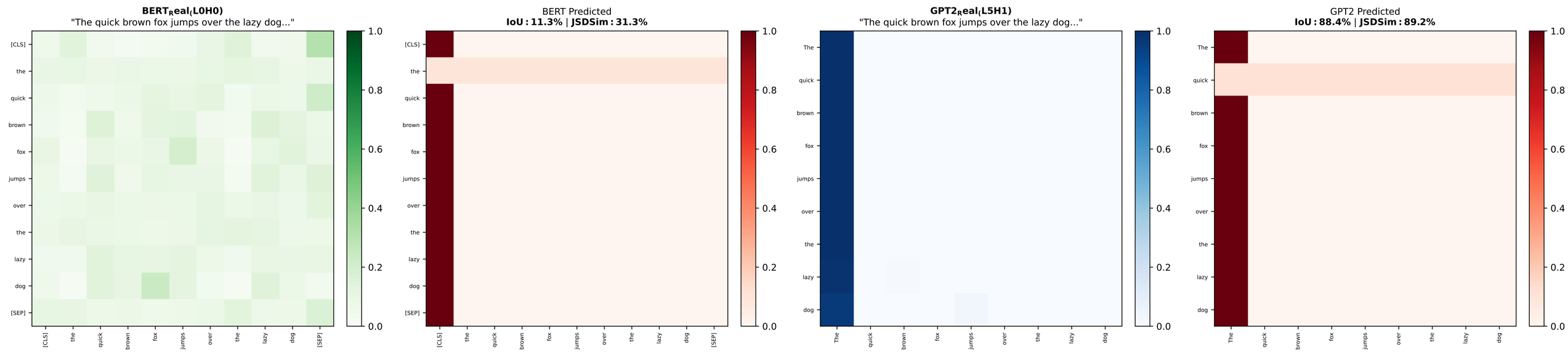
Pattern: dependencies

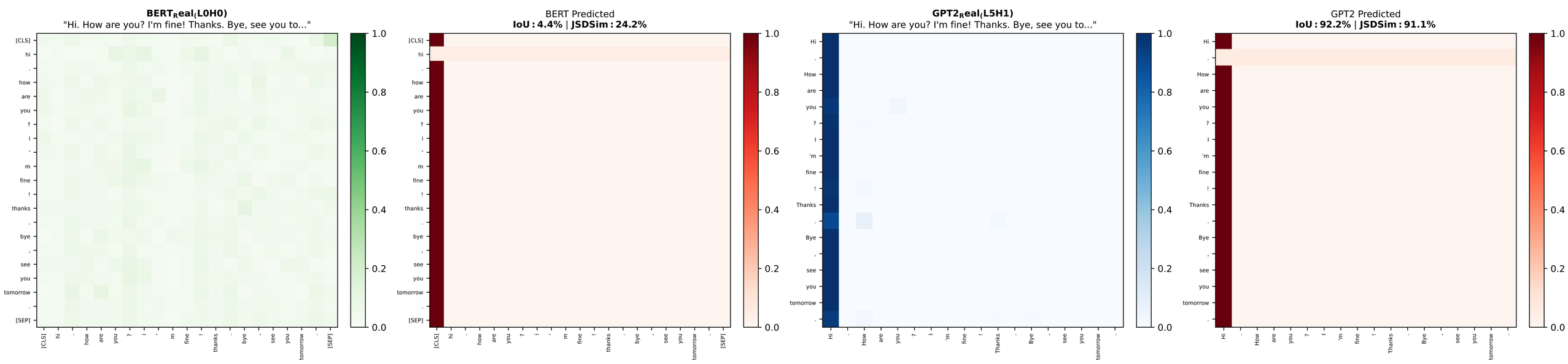
```
def dependencies(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
                                words = sentence.split()
                               doc = nlp(" ".join(words))
                                  check_errors = False
                                    if check_errors:
                         if len(doc) == 0: print("problem, doc empty")
     if len(doc) != (len_seq-2): print("problem, doc length mismatch", len(doc), len(toks)-2)
                                    for stok in doc:
                                   parent_index = stok.i
                             for child_stok in stok.children:
                                  child_index = child_stok.i
                            out[parent_index+1, child_index+1] = 1
                            out[child_index+1, parent_index+1] = 1
                                     out[0, 0] = 1
                                     out[-1, 0] = 1
                                      out += 1e-4
                         out = out / out.sum(axis=1, keepdims=True)
                          return "Dependency Parsing Pattern", out
```

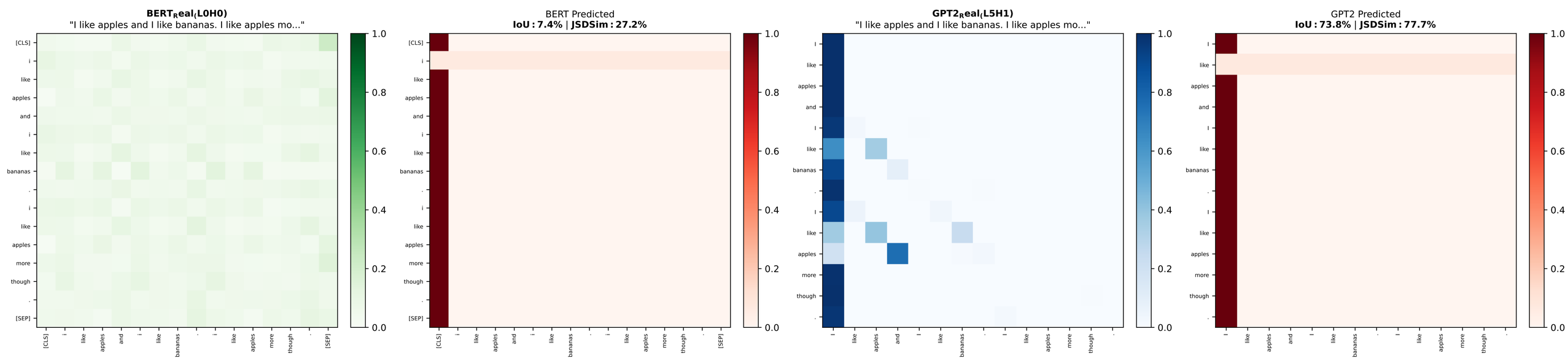


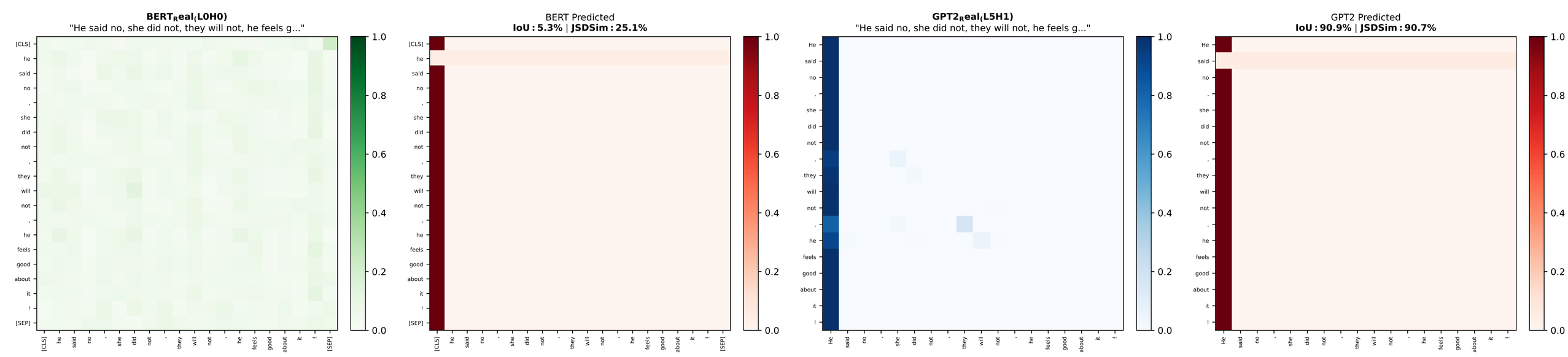
Pattern: dominant_subject_attention

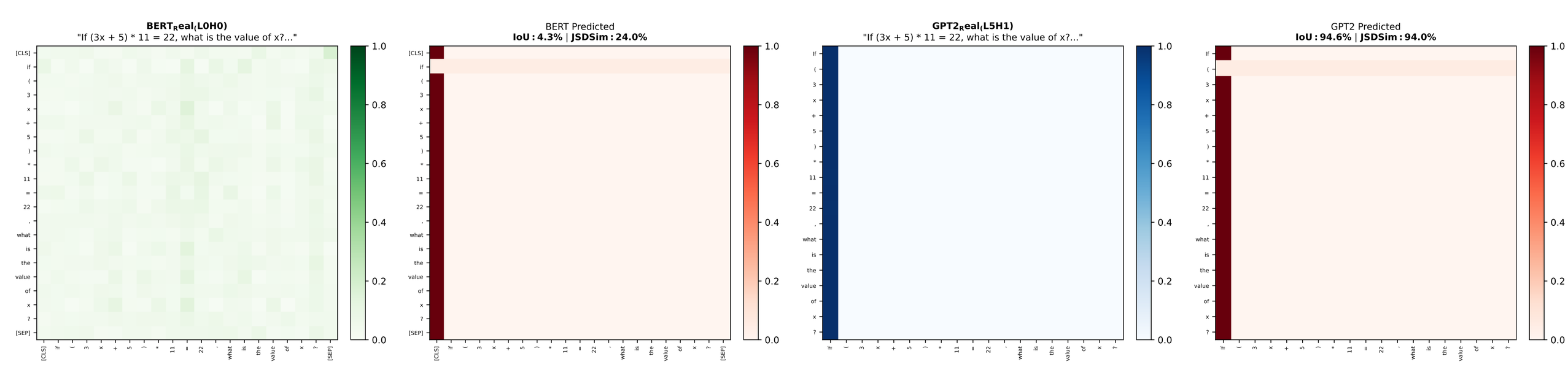
```
def dominant_subject_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Identify first non-special token as dominant subject
    dominant_subject_idx = 1

    # Create an attention pattern matrix
    out[dominant_subject_idx, :] = 1 # Dominant subject attends to all tokens
    out[:, 0] = 1  # [CLS] token receives some attention, assuming CLS-like functionality
    out[-1, 0] = 1  # [SEP] token similar functionality

    # Normalize
    out = out / out.sum(axis=1, keepdims=True)

    return "Dominant Subject Attention Pattern", out
```





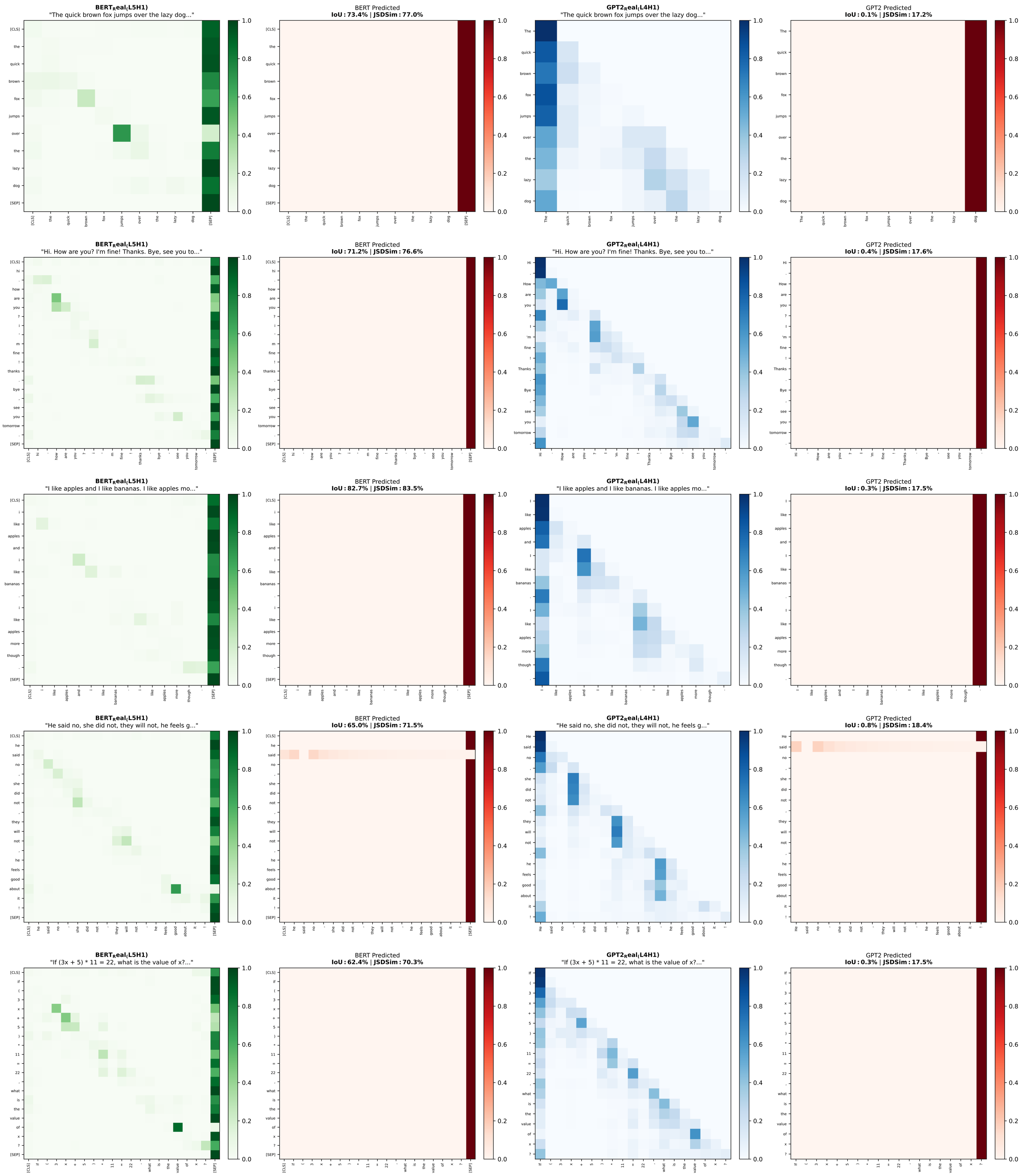






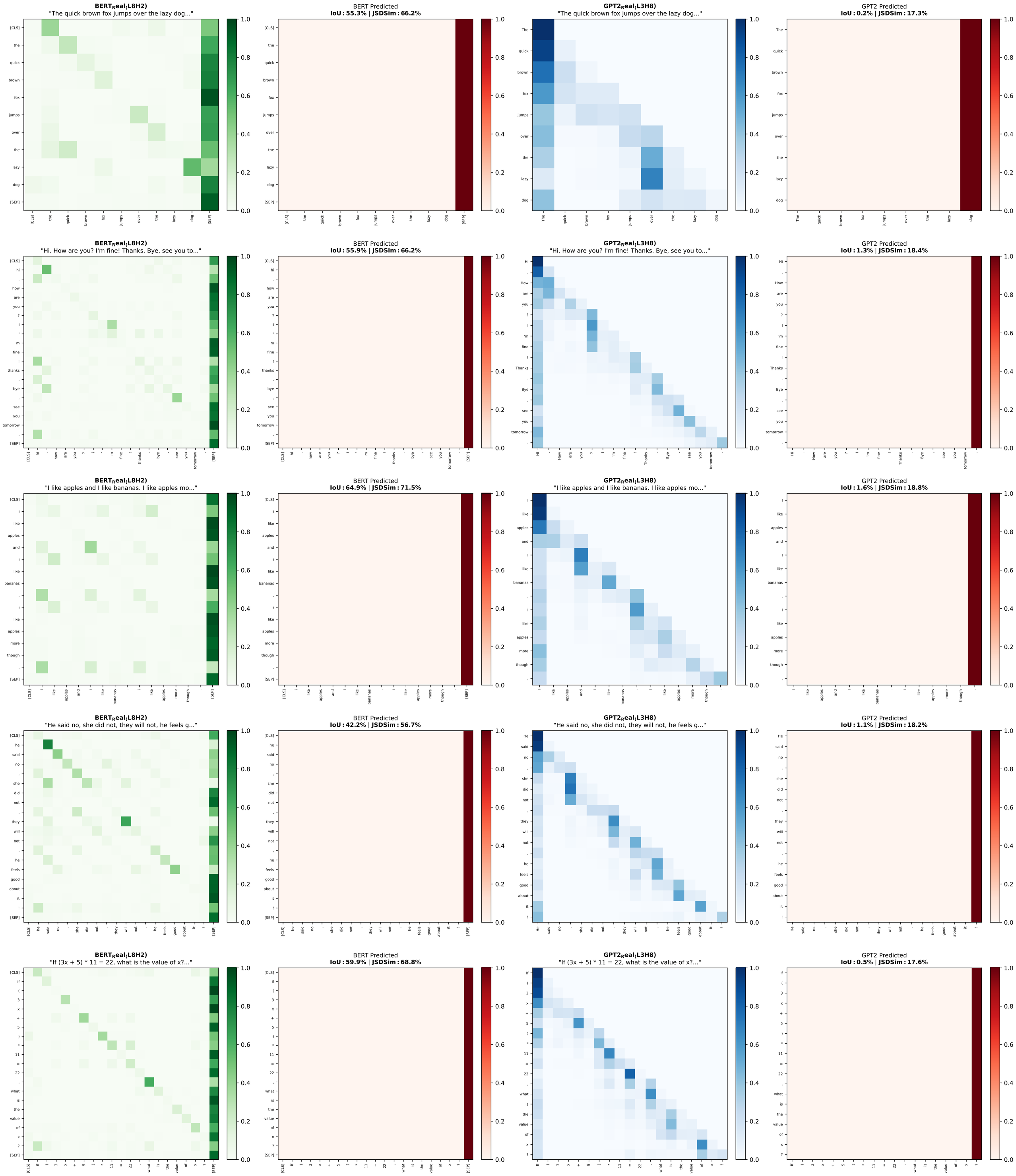
Pattern: emphasize_verbs_and_objects

```
def emphasize_verbs_and_objects(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    words = tokenizer.convert_ids_to_tokens(toks.input_ids[0])

    # keywords to focus that are associated with action
action_keywords = ["found", "difficult", "share", "went", "said", "sew", "smiled", "sharing", "fixing"]

    for i, word in enumerate(words):
    # Assuming the word is an action if it matches the keywords
        if any(kw in word for kw in action_keywords):
            for j in range(len_seq):
                if i != j:
                out[i, j] = 1 / (abs(i - j) + 1)  # Closer tokens get more attention

    for row in range(len_seq):
        # Ensure no row is all zeros
        if out[row].sum() == 0:
            out[row, -1] = 1.0

    out += 1e-4  # Avoid division by zero
out = out / out.sum(axis=1, keepdims=True)  # Normalize by rows

    return "Emphasizing Action Verbs and Their Objects", out
```



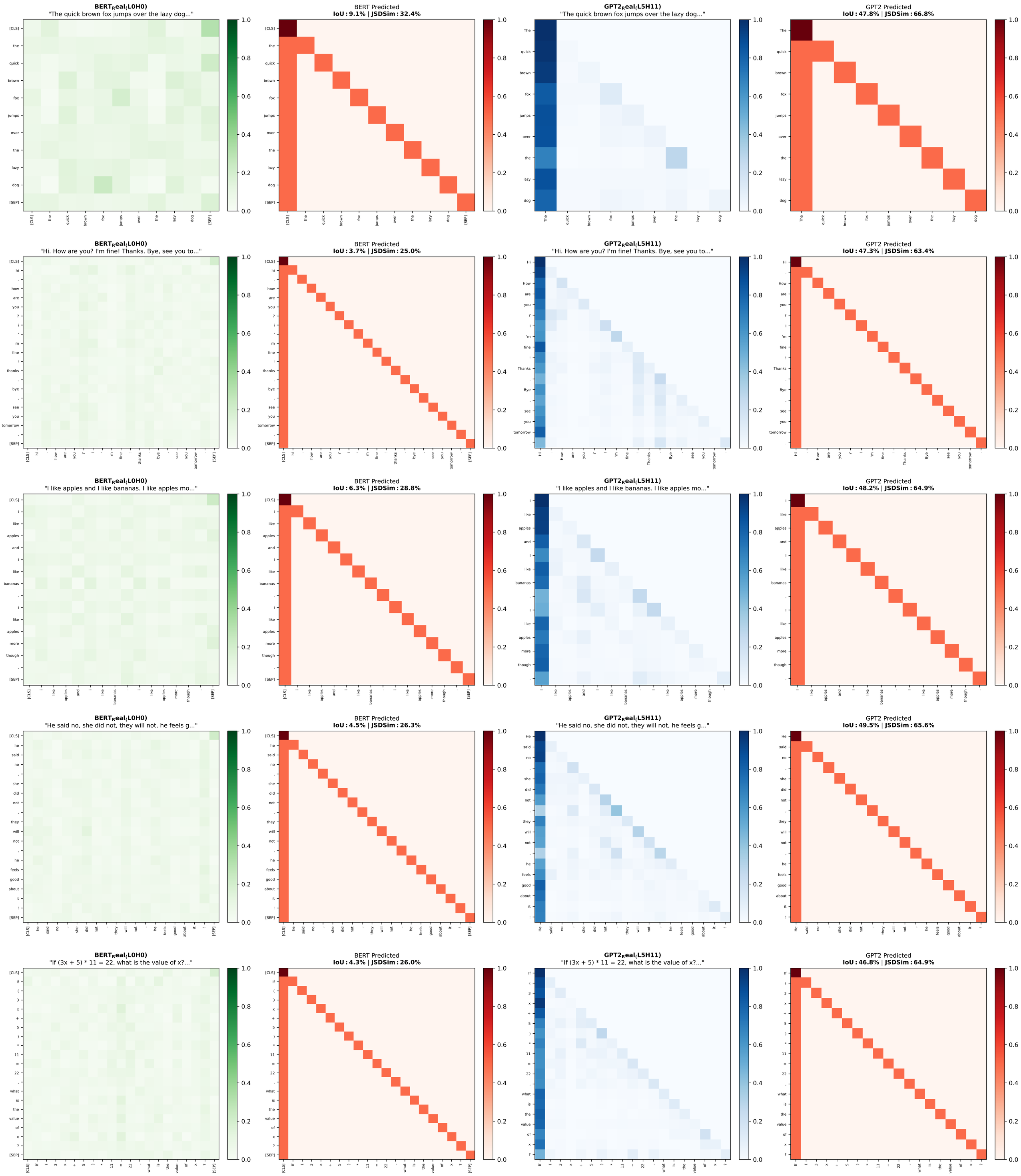
Pattern: eos_attention

```
def eos_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    out[:, -1] = 1
    return "EOS Pattern", out
```



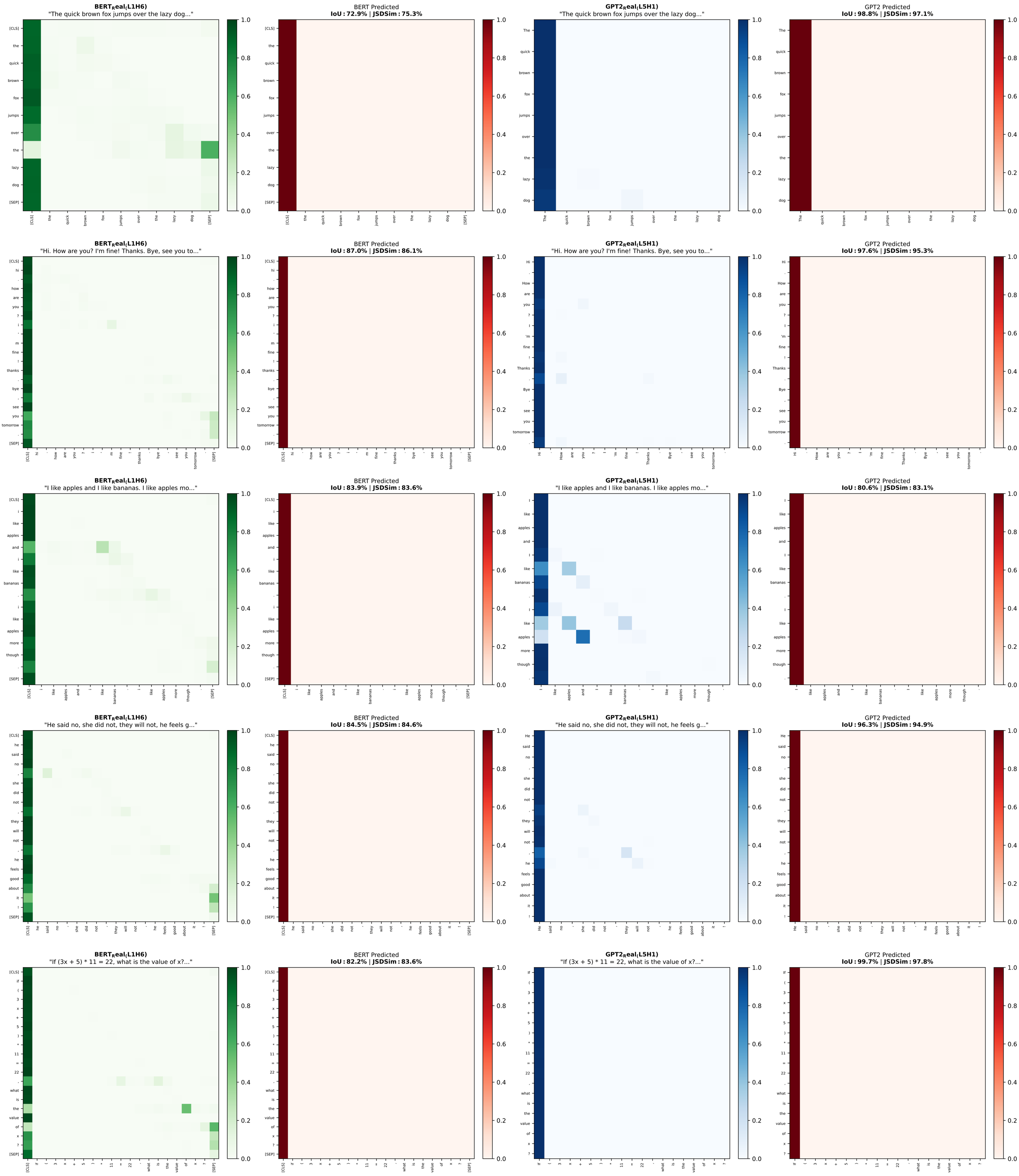
Pattern: first_token_dominance

```
def first_token_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
          # The first token heavily impacts the majority of other tokens
                                  out[1:, 0] = 1
                                 # Self-attention
                         for i in range(len_seq):
                                    out[i, i] = 1

        # cls (out[0,0]) and eos (out[-1,0]) are specialized self_attentions
                                   out[0, 0] = 1
                                  out[-1, 0] = 1
                                    # Normalize
                                    out += 1e-4
                     out = out / out.sum(axis=1, keepdims=True)
                        return "First Token Dominance", out
```



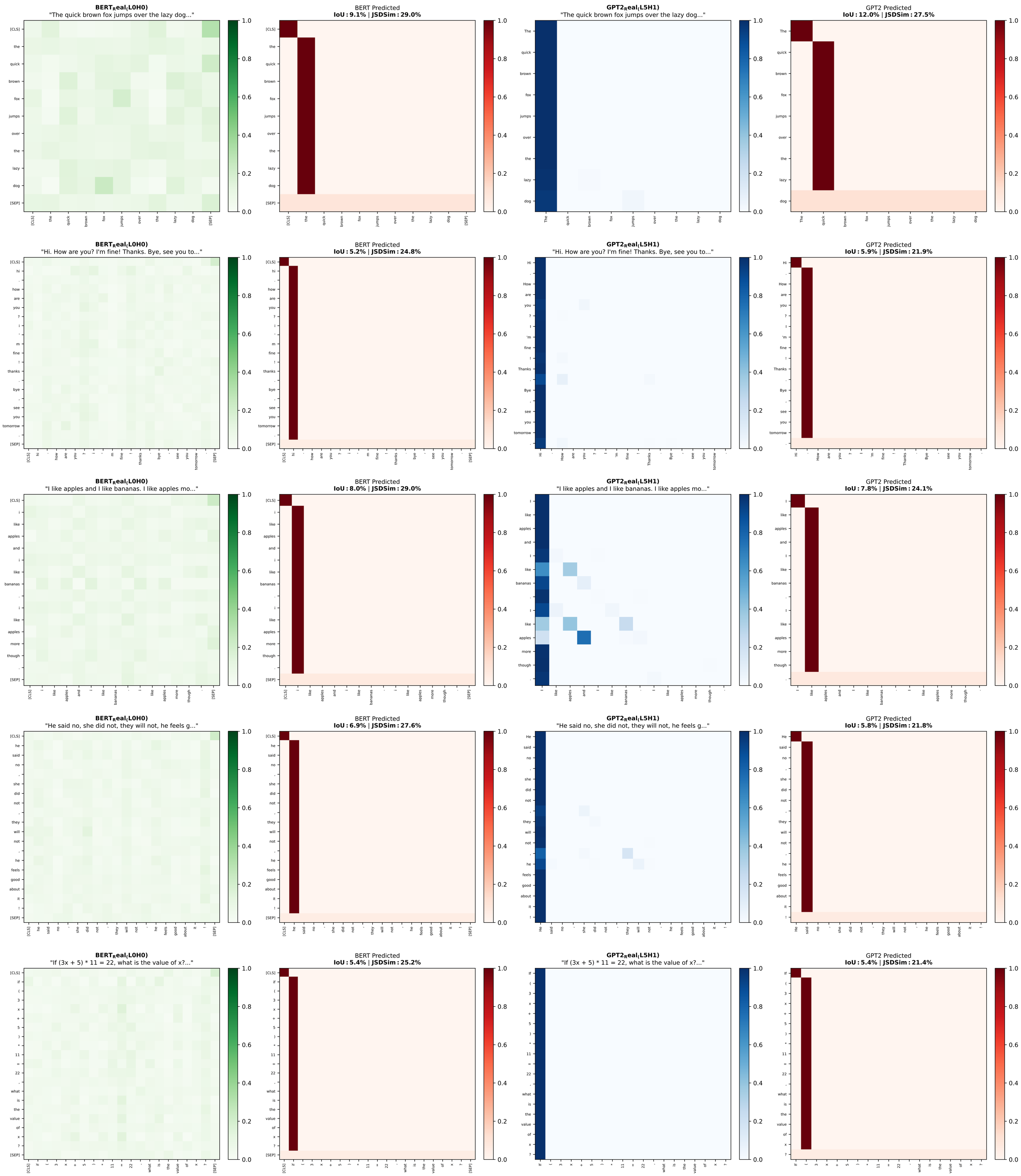
Pattern: first_token_domination

```
def first_token_domination(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # The first token (index 0) attention dominates over other tokens.
    for i in range(len_seq):
        if i != 0:
            out[i, 0] = 1.0
        else:
            out[i, i] = 1.0 # Self-attention for the first token

    # Ensure no row is all zeros
    for row in range(len_seq):
        if out[row].sum() == 0:
            out[row, -1] = 1.0

    out = out / out.sum(axis=1, keepdims=True)  # Normalize
    return "First Token Domination", out
```



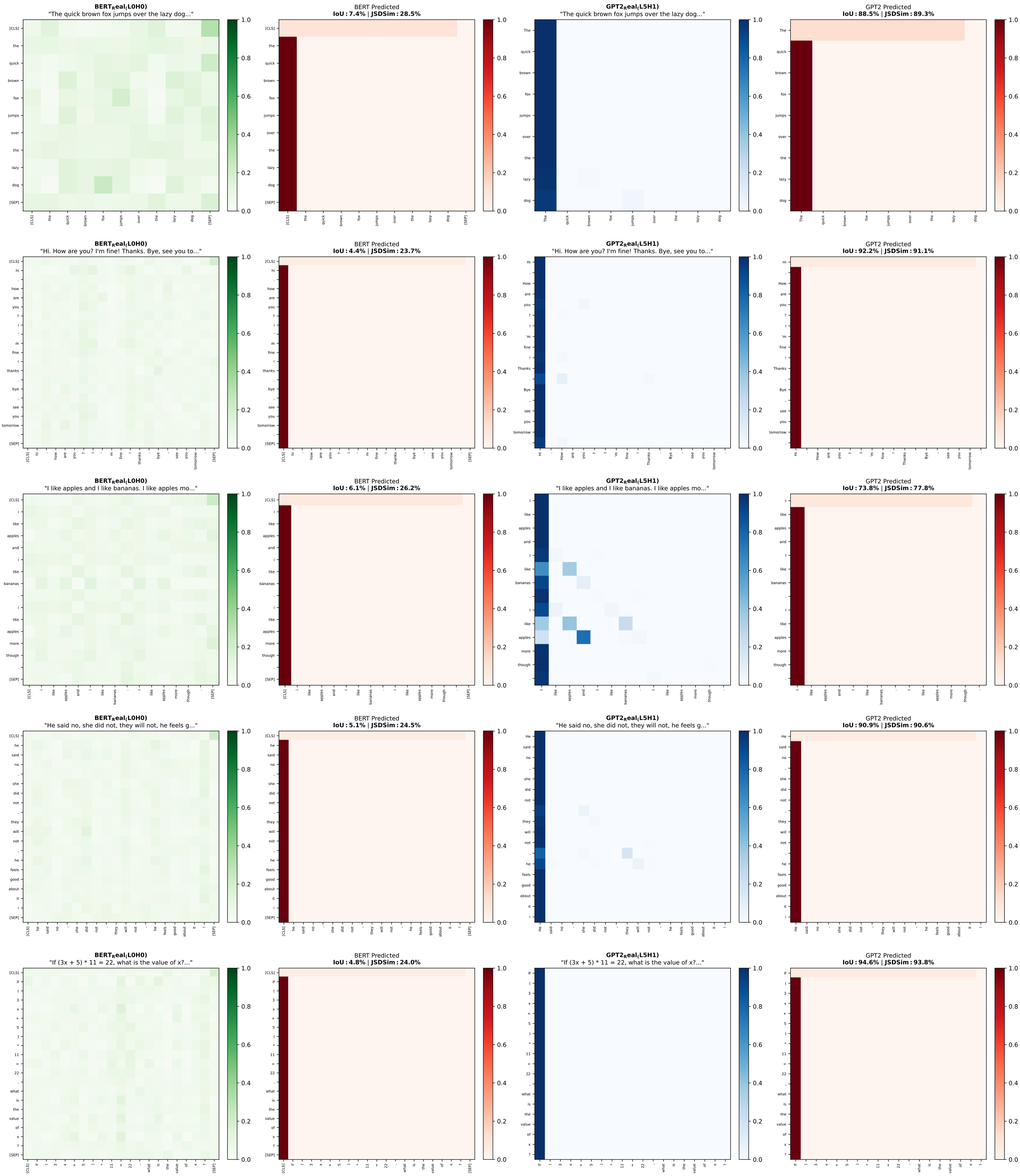
Pattern: first_token_emphasis

```
def first_token_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                        len_seq = len(toks.input_ids[0])
                                       out = np.zeros((len_seq, len_seq))

                    # Emphasize attention from all tokens to the first actual token
            for i in range(1, len_seq-1):  # Ignoring special tokens usually at 0 and len_seq-1
             out[i, 1] = 1.0  # Attention directed towards the first token after the special [CLS]

                                       # Special token [CLS] self-attends
                                                out[0, 0] = 1.0

                                  # Ensure every row sums to 1 by normalizing
                                     out += 1e-4  # Avoid division by zero
                                   out = out / out.sum(axis=1, keepdims=True)

                                       return "First Token Emphasis", out
```



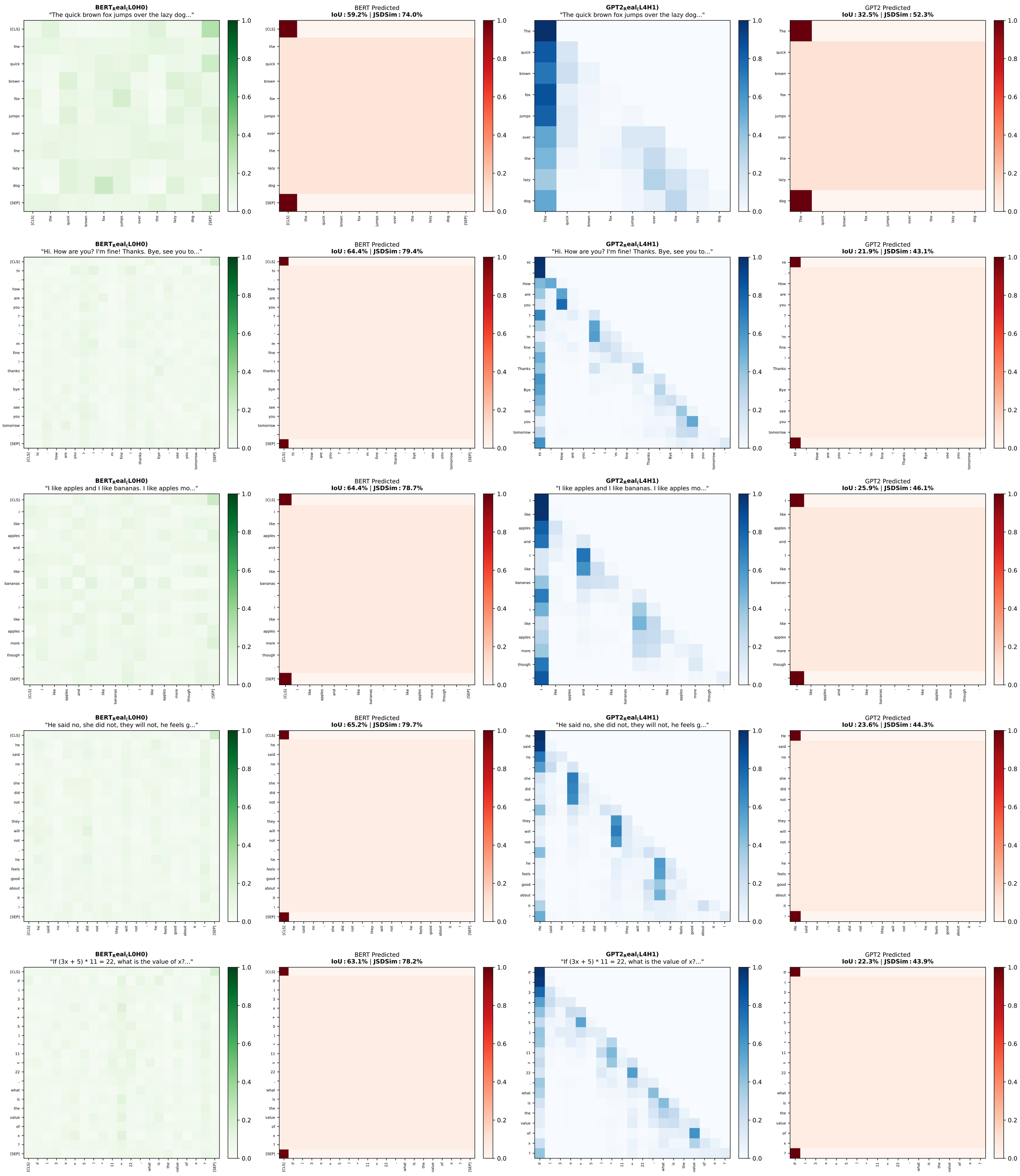
Pattern: head_initial_token_emphasis

```
def head_initial_token_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Focus on the first token and distribute its attention based on apparent syntactic roles observed
    for i in range(1, len_seq - 1):
    # Assign high attention to the first token and distribute attention to tokens based on syntactic positions
    out[0, i] = 1  # The first token, which tends to be the subject or important opening, holds attention
    out[i, 0] = 1  # Mutual attention back

    # Ensure CLS and EOS token self-attend, if applicable (not present in GPT-2)
    out[0, 0] = 1
    out[-1, 0] = 1

    # Normalize attention row-wise
    out = out / out.sum(axis=1, keepdims=True)

    return "Head Initial Token Emphasis with Syntactic Role Consistency", out
```



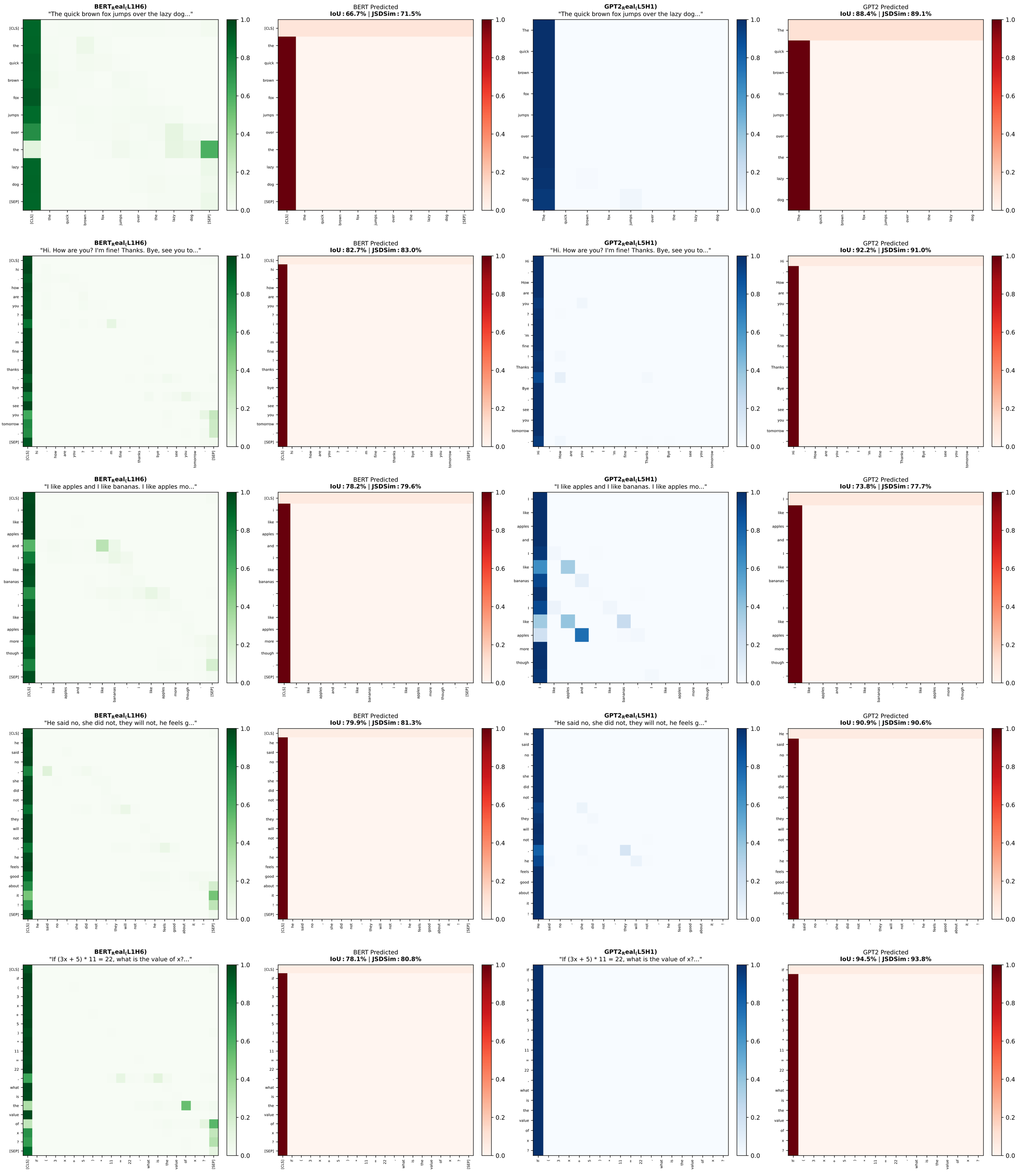
Pattern: high_saliency_relationship_detection

```
def high_saliency_relationship_detection(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    # Extract key tokens that match criteria: suffixes "ly", root anticipation, sentiment descriptors.
    high_attention_terms = [
        "##s", "hue", "##ly", "unexpected", "complex", "ness", "scent",
        "y", "ness", "surge", "excitement", "slowly", "delicate", "delicious"
    ]
    tokens = toks.tokens()[0]
    token_index_map = {i: tokens[i] for i in range(len(tokens))}
    # Mark salient terms in relation to their most similar counterpart of high attention terms.
    for idx, token in token_index_map.items():
        if any(term in token for term in high_attention_terms):
            for j in range(len_seq):
                if j != idx:
                    out[idx, j] = (1 / len_seq)  # Assign lower weight by factor of length to all other tokens.
            out[idx, idx] = 1  # Highest weight for the term itself.
    out[0, 0] = 1
    out[-1, 0] = 1
    out += 1e-4
    out = out / out.sum(axis=1, keepdims=True)  # Normalize attention matrix rows.
    return "High Saliency Relationship Detection", out
```



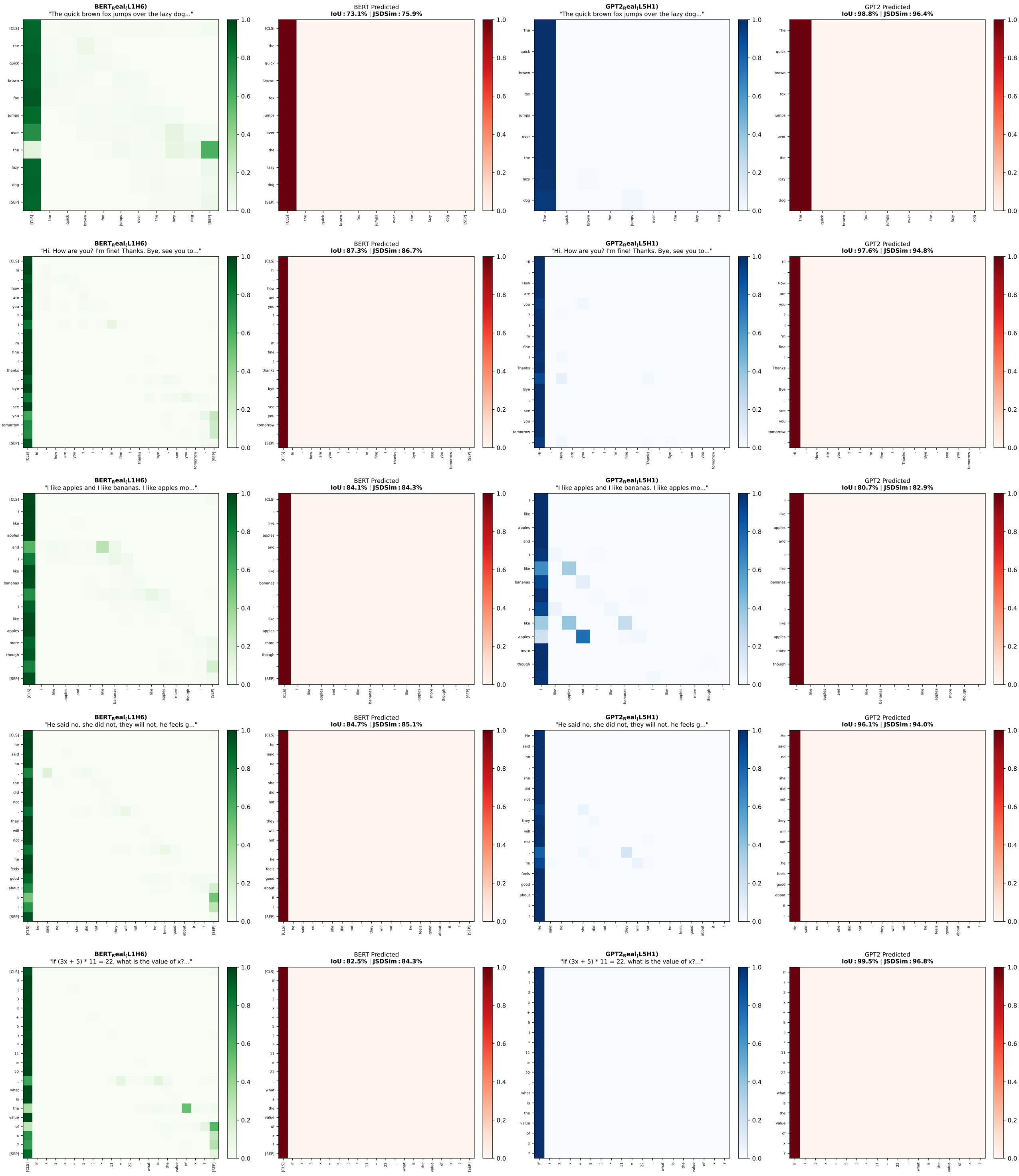
Pattern: initial_contextual_attention

```
def initial_contextual_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Assume tokenization is consistent and we can assume alignment of indices
    # Initialize the pattern
    for i in range(len_seq):
        for j in range(len_seq):
            # Implement the pattern of high attention from the first token to all others
            if i == 0:
                out[i, j] = 1
# Implement the pattern of stem token having higher weight to the content words it dominates (ex: in 'The sun dipped below', sun would dominate 'dipped below')
            elif j == 0:
                out[i, j] = 0.1
            else:
                out[i, j] = 0

    # Normalize out matrix by rows
    row_sums = out.sum(axis=1, keepdims=True)
    for i in range(len_seq):
        if row_sums[i] == 0:
            out[i, -1] = 1
        else:
            out[i] /= row_sums[i]

    return "Initial Contextual Attention", out
```



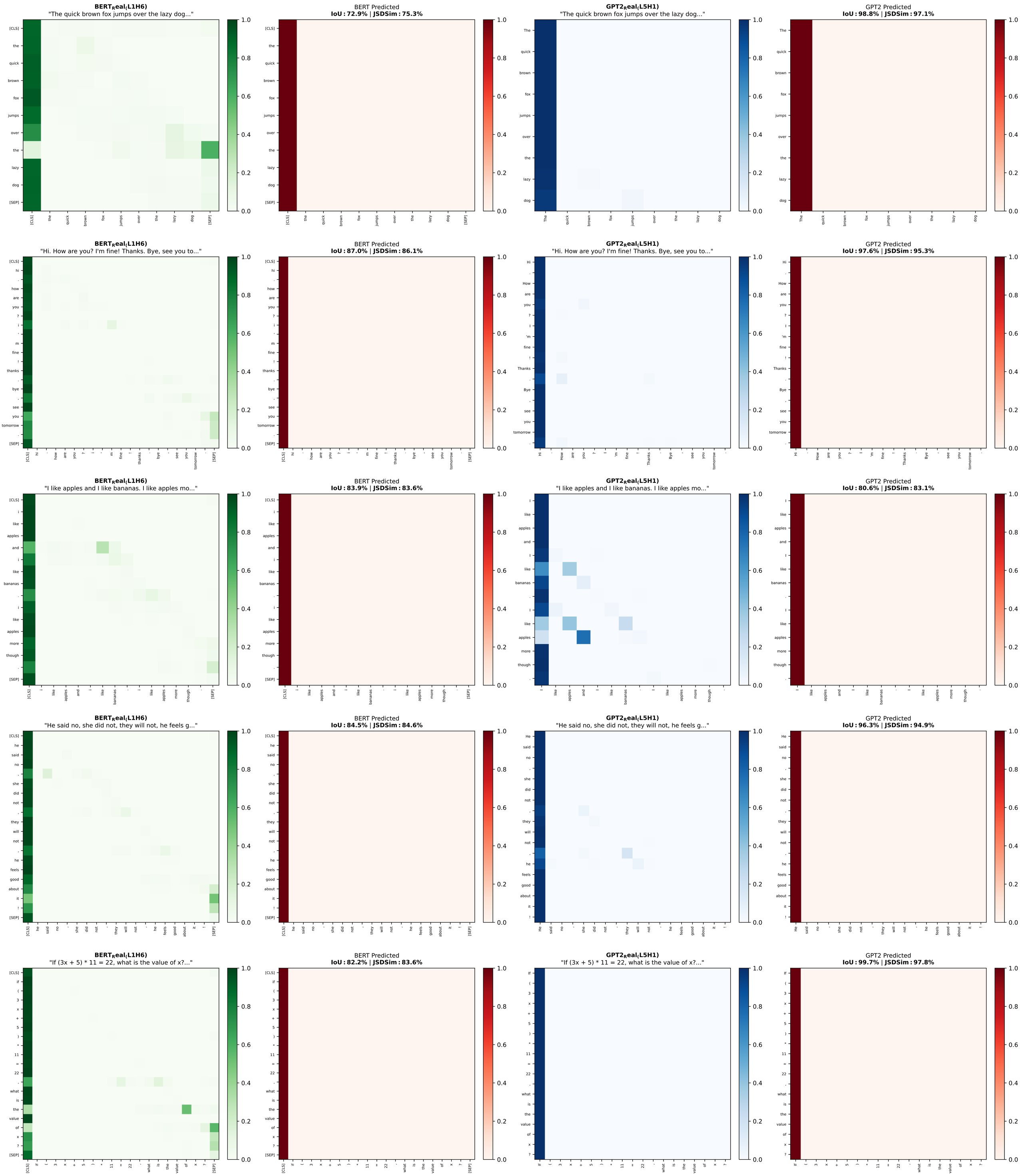
Pattern: initial_element_reinforcement

```
def initial_element_reinforcement(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
                                    toks = tokenizer([sentence], return_tensors='pt')
                                            len_seq = len(toks.input_ids[0])
                                           out = np.zeros((len_seq, len_seq))
                                                words = sentence.split()
                                               doc = nlp(" ".join(words))

                            # Build alignment between tokenizer and spacy tokens/words
                                               token_to_word_mapping = {}
                                                       index = 0
                                                    for tok in doc:
                                                     if tok.is_space:
                                                          continue
                    for _ in tokenizer([' ' + tok.text], return_offsets_mapping=True)['offset_mapping'][0]:
                                               token_to_word_mapping[index] = tok
                                                           index += 1

                         # Initial token strongly attends to itself and other tokens attend to it
                                 initial_token_id = list(token_to_word_mapping.keys())[0]
                                                for i in range(len_seq):
                                                  if i == initial_token_id:
                                            out[i, i] = 1  # Self-attention with high weight
                                                          else:
                        out[i, initial_token_id] = 0.7  # Other tokens have attention to the initial token

                                   # Normalize out matrix to ensure valid probabilities
                                                for row in range(len_seq):
                                             out[row] += 1e-4  # Avoid division by zero
                                   out[row] = out[row] / out[row].sum()  # Normalize by row sum

                                 return "Initial Element Reinforcement with Intra-Sentence Reference", out
```



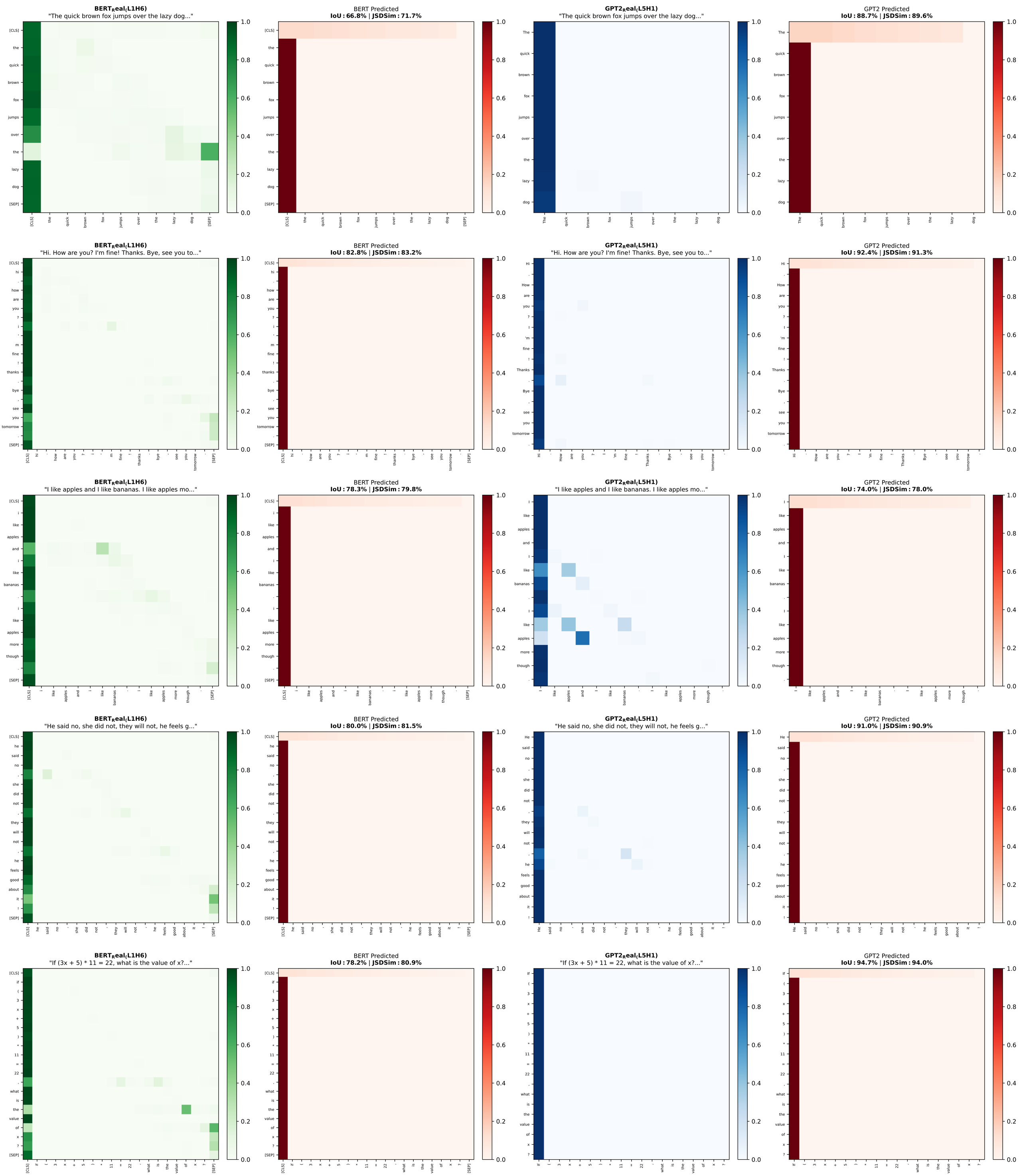
Pattern: initial_phrase_contextualization

```
def initial_phrase_contextualization(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # The first token always attends to itself while all other tokens focus on the initial part of the sentence.
    attention_threshold = 0.7

    for i in range(1, len_seq):
        if i <= int(len_seq * attention_threshold):
            # Emphasizing strong attention to the first token among the first part tokens
            out[i, 0] = 1
        else:
            # Remaining tokens have reduced focus
            out[i, 0] = 0.5

    # First token has self-attention
    out[0, 0] = 1

    # Normalize by row
    out /= out.sum(axis=1, keepdims=True)
    return "Initial Phrase Contextualization", out
```



Pattern: initial_phrase_dominance

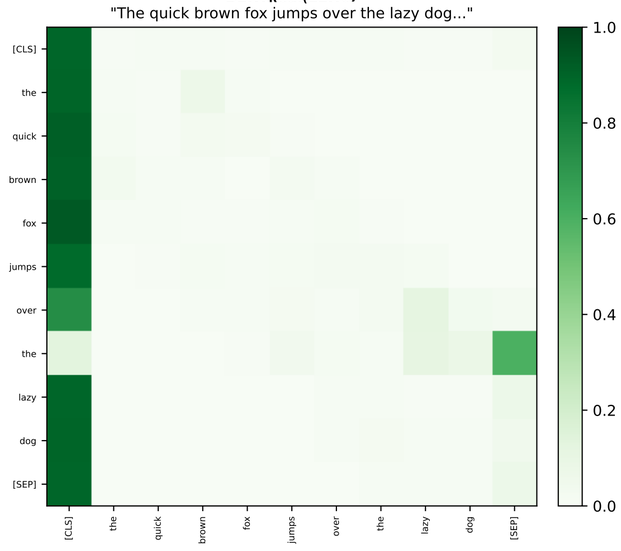
```
def initial_phrase_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
                    toks = tokenizer([sentence], return_tensors="pt")
                            len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))

     # Hypothesis: Head focuses heavily on the initial phrase and distributed attention
            # Calculate attention weight based on the position in the sequence
                                  initial_weight = 1.0
       decay_factor = 0.9  # Decay factor for attention spread from the initial token

        # Assign attention based on the decay pattern starting from the first token
                                for i in range(1, len_seq-1):
                        out[0, i] = initial_weight * (decay_factor ** (i-1))
                        out[i, 0] = initial_weight * (decay_factor ** (i-1))

            # Ensure CLS token at pos 0 and EOS token at -1 have self-attention
                                     out[0, 0] = 1
                                    out[-1, 0] = 1

                                # Normalize the attention matrix
                          out = out / out.sum(axis=1, keepdims=True)
                            return "Initial Phrase Dominance", out
```



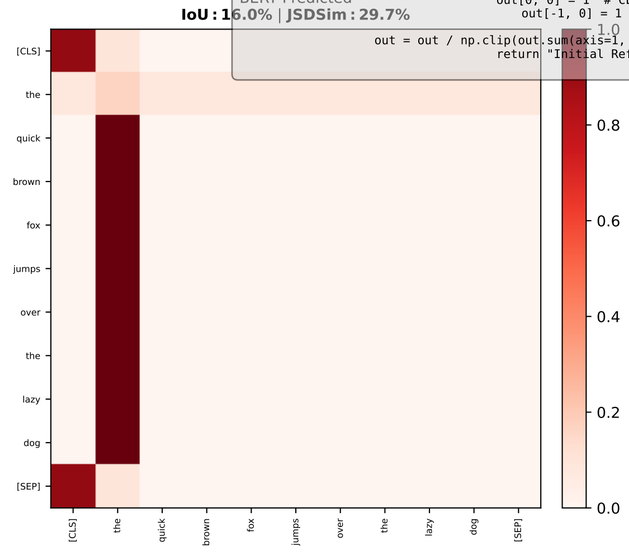
Pattern: initial_reference_attention

```
def initial_reference_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    words = sentence.split()

    # Heuristic: Find indices of the initial subject or noun phrase
    # This is done by finding the first non-determiner (e.g., not "The", "A") token
    # and gathering following tokens until a verb or punctuation is encountered.
    start_idx = None
    for i, word in enumerate(words):
        if word.lower() not in {'the', 'a', 'an', ',', '.', ':', ';'}:
            start_idx = i
            break
    if start_idx is None:
        start_idx = 0

    # Assuming the subject or main noun phrase ends at either a punctuation or conjunction
    end_idx = start_idx
    for i, word in enumerate(words[start_idx:], start_idx):
        if word.endswith(('.', ',', ':', ';', '?', '!', 'and', 'or', 'but')):
            end_idx = i
            break

    # Collect attention on the initial subject or noun phrase
    for i in range(start_idx + 1, end_idx + 1):
        out[i, start_idx] = 1
        out[start_idx, i] = 1

    # Give attention to all tokens linking back to the initial phrase
    for i in range(len_seq):
        out[i, start_idx] += 0.1
        out[start_idx, i] += 0.1

    out[0, 0] = 1  # CLS token self-attention
    out[-1, 0] = 1  # EOS token to CLS
    out = out / np.clip(out.sum(axis=1, keepdims=True), a_min=1e-9, a_max=None)
    return "Initial Reference Attention", out
```

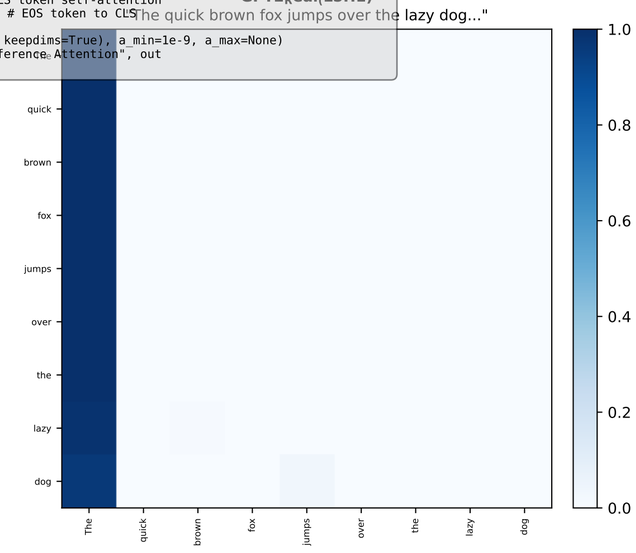
BERT$_{R}$eal$_{(}$L1H6)
"The quick brown fox jumps over the lazy dog..."



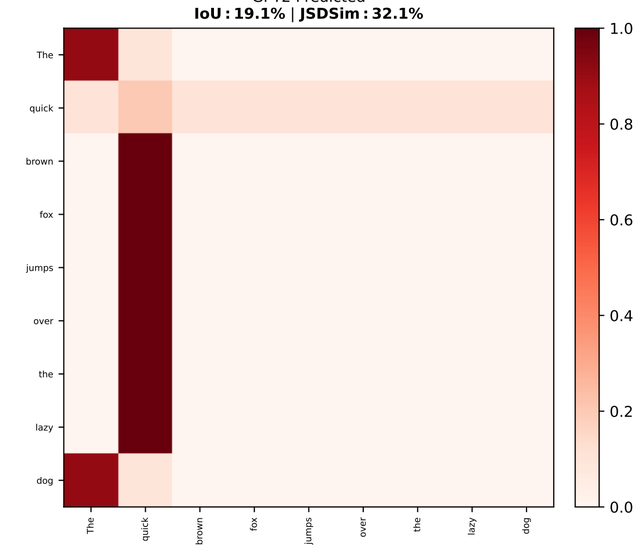
BERT Predicted
IoU : 16.0% | JSDSim : 29.7%



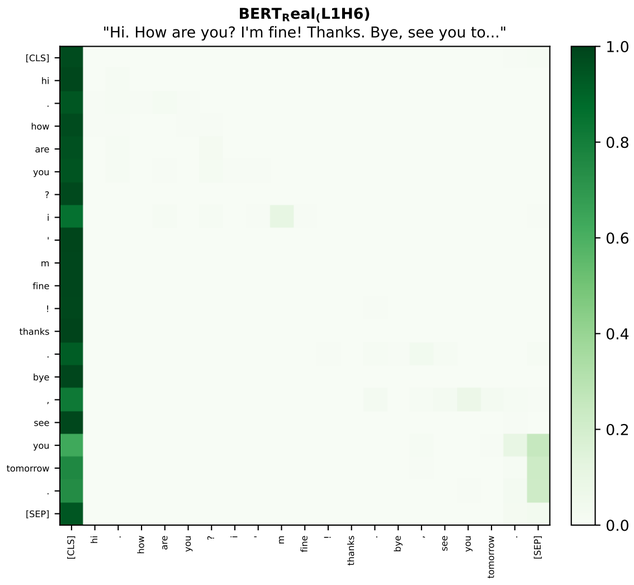
GPT2$_{R}$eal$_{(}$L5H1)
"The quick brown fox jumps over the lazy dog..."



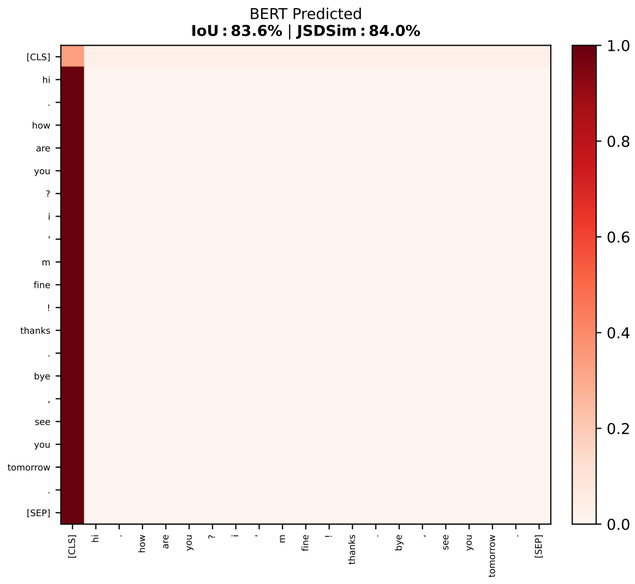
GPT2 Predicted
IoU : 19.1% | JSDSim : 32.1%



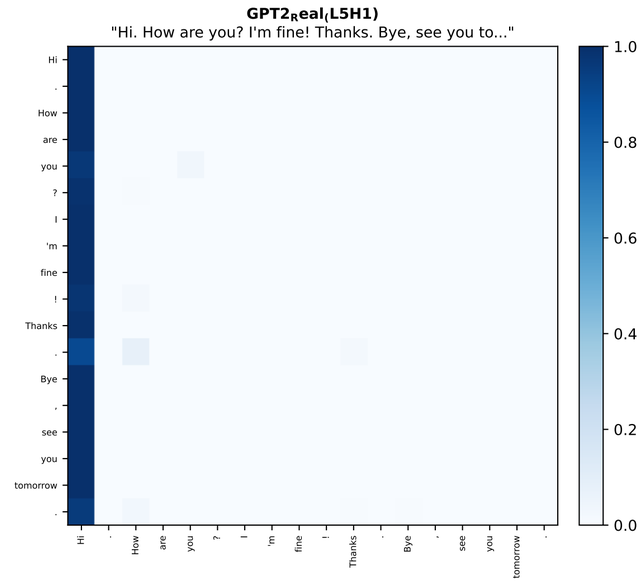
BERT$_{R}$eal$_{(}$L1H6)
"Hi. How are you? I'm fine! Thanks. Bye, see you to..."



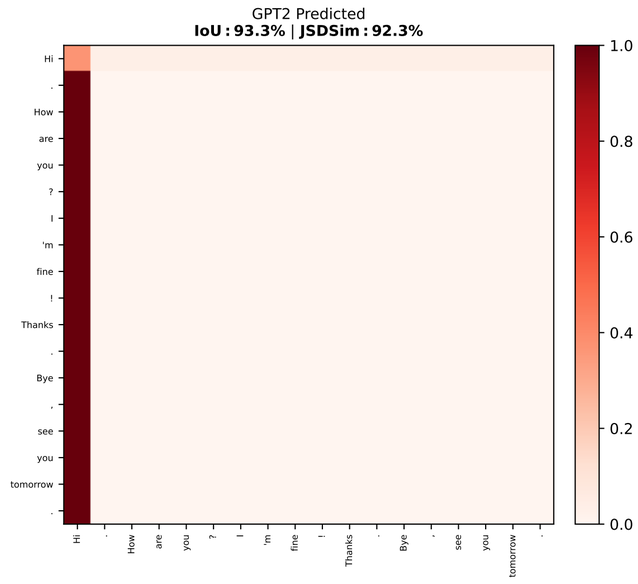
BERT Predicted
IoU : 83.6% | JSDSim : 84.0%



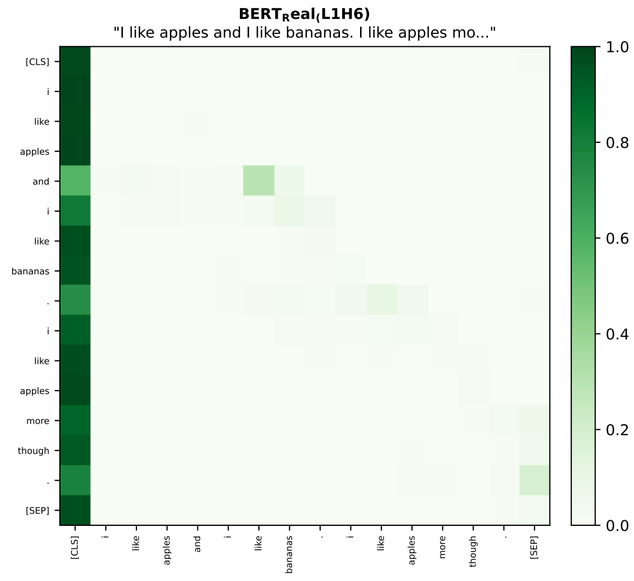
GPT2$_{R}$eal$_{(}$L5H1)
"Hi. How are you? I'm fine! Thanks. Bye, see you to..."



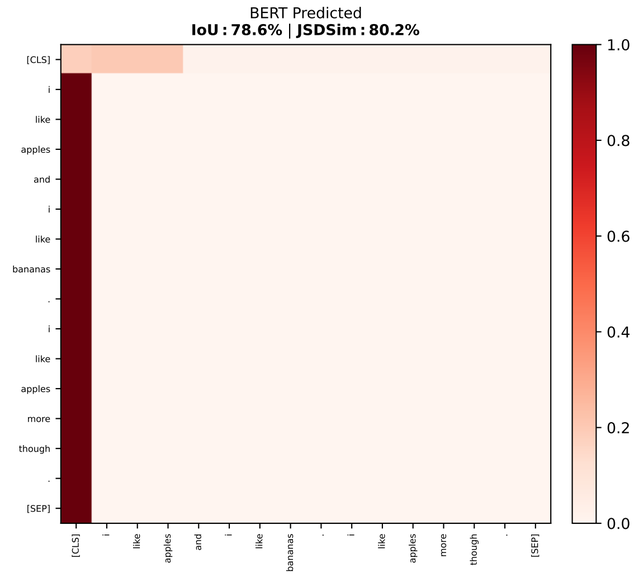
GPT2 Predicted
IoU : 93.3% | JSDSim : 92.3%



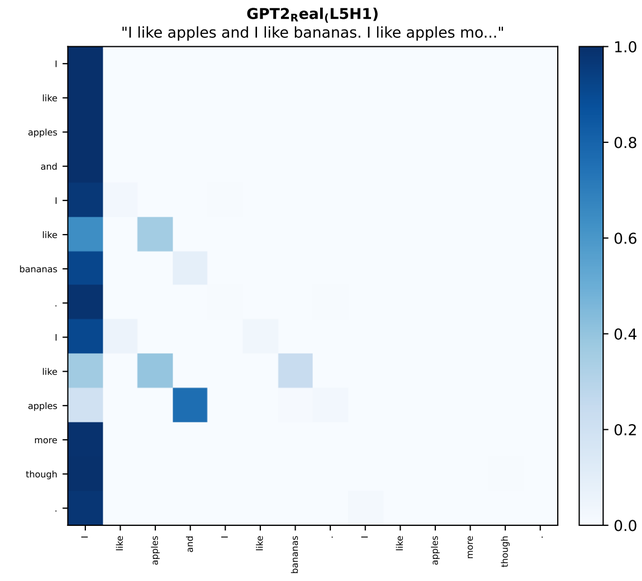
BERT$_{R}$eal$_{(}$L1H6)
"I like apples and I like bananas. I like apples mo..."



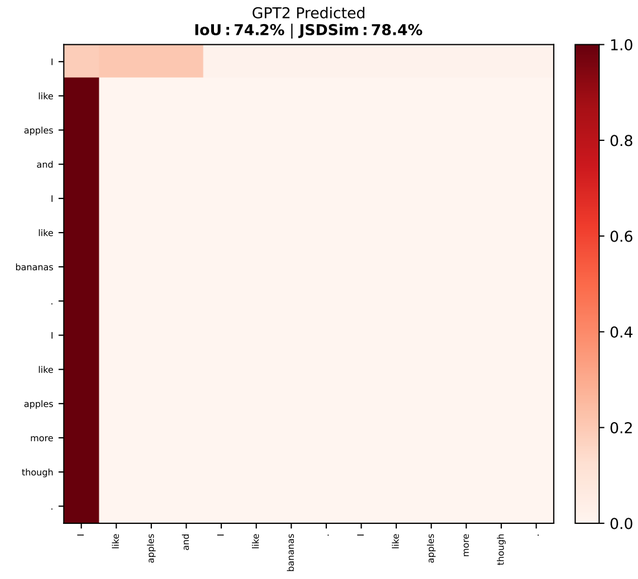
BERT Predicted
IoU : 78.6% | JSDSim : 80.2%



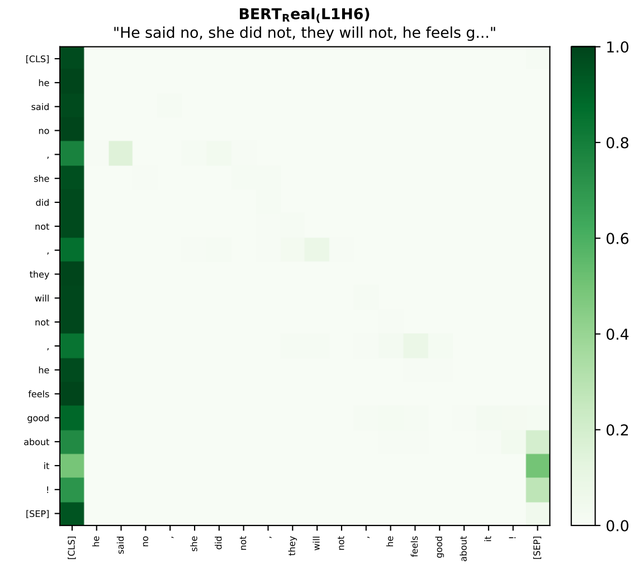
GPT2$_{R}$eal$_{(}$L5H1)
"I like apples and I like bananas. I like apples mo..."



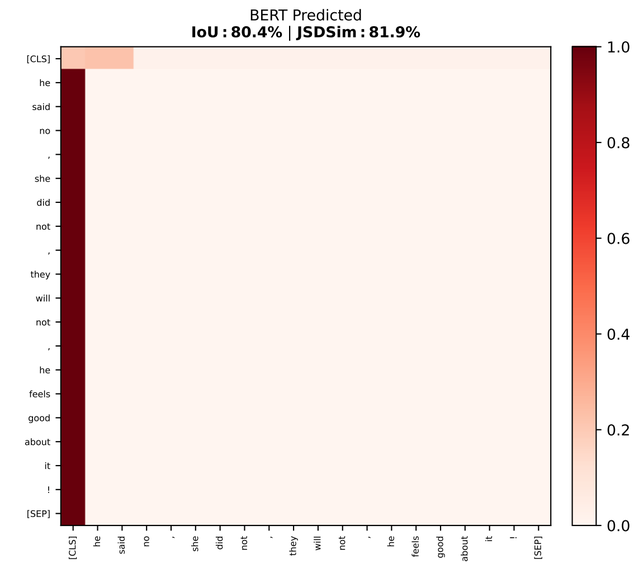
GPT2 Predicted
IoU : 74.2% | JSDSim : 78.4%



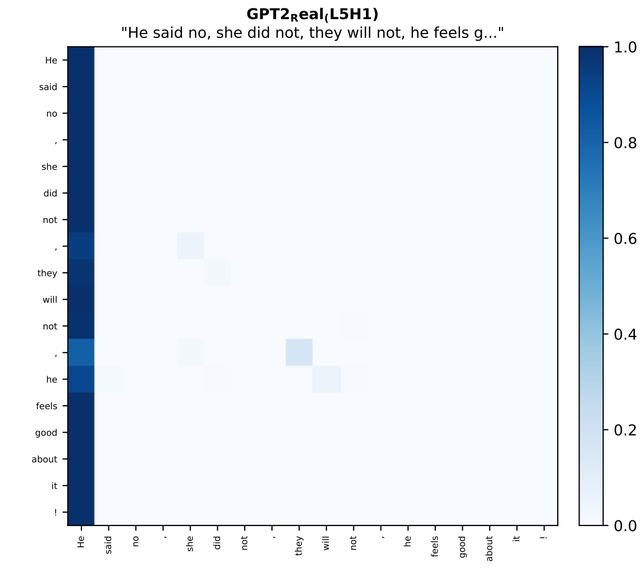
BERT$_{R}$eal$_{(}$L1H6)
"He said no, she did not, they will not, he feels g..."



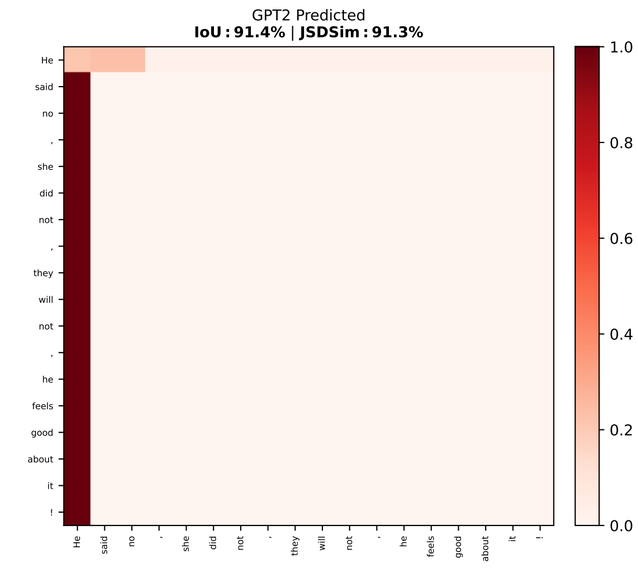
BERT Predicted
IoU : 80.4% | JSDSim : 81.9%



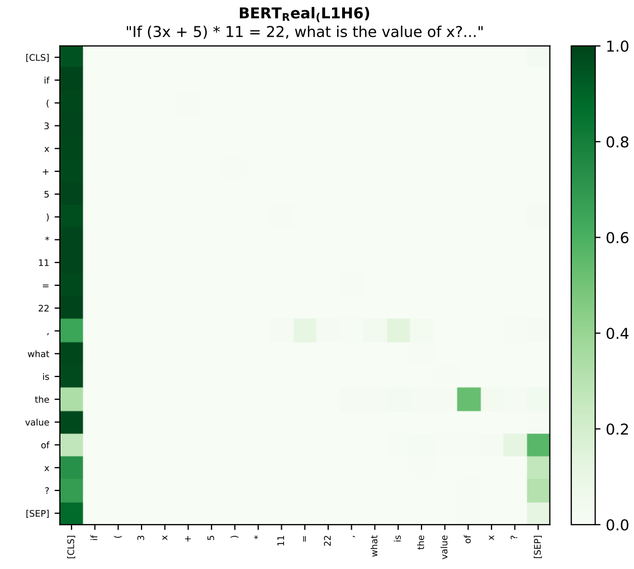
GPT2$_{R}$eal$_{(}$L5H1)
"He said no, she did not, they will not, he feels g..."



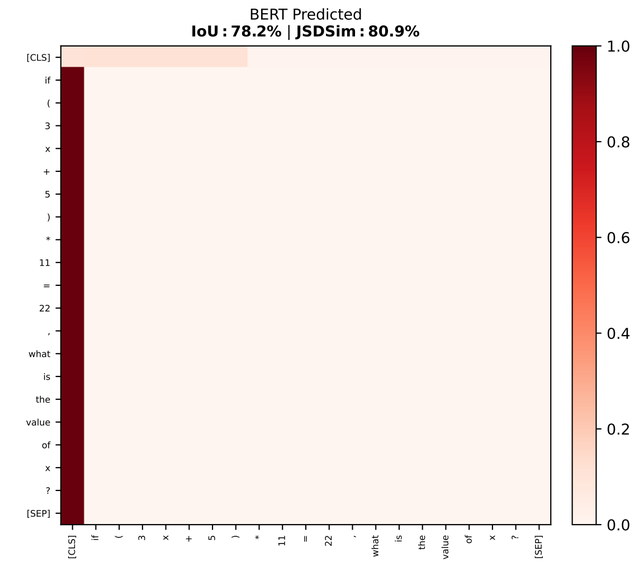
GPT2 Predicted
IoU : 91.4% | JSDSim : 91.3%



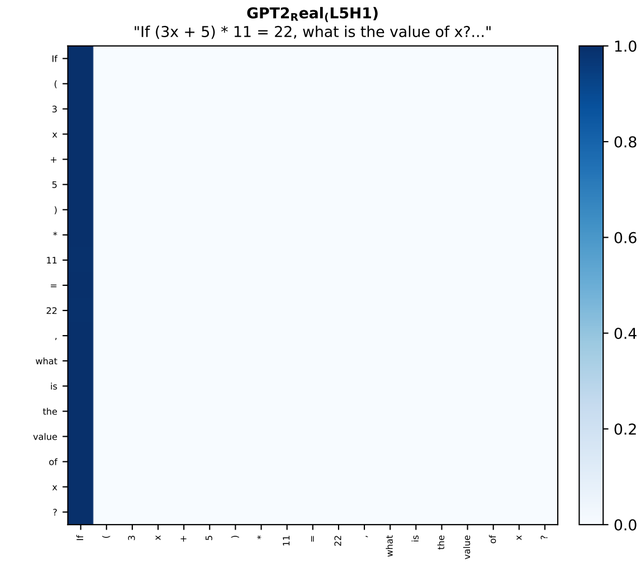
BERT$_{R}$eal$_{(}$L1H6)
"If (3x + 5) * 11 = 22, what is the value of x?..."



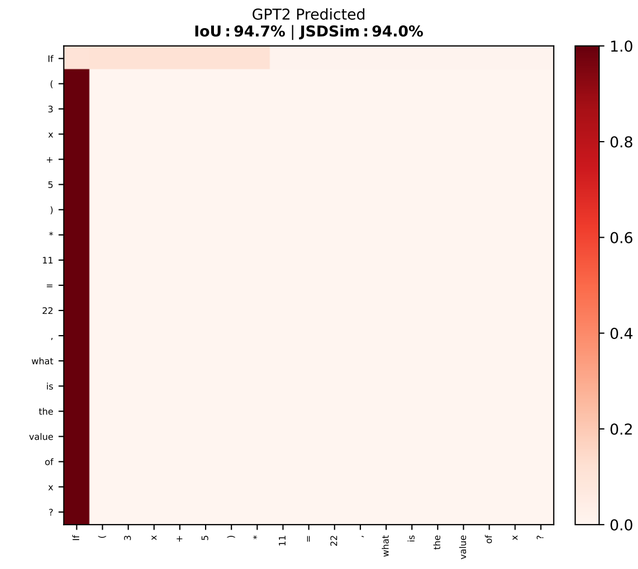
BERT Predicted
IoU : 78.2% | JSDSim : 80.9%



GPT2$_{R}$eal$_{(}$L5H1)
"If (3x + 5) * 11 = 22, what is the value of x?..."

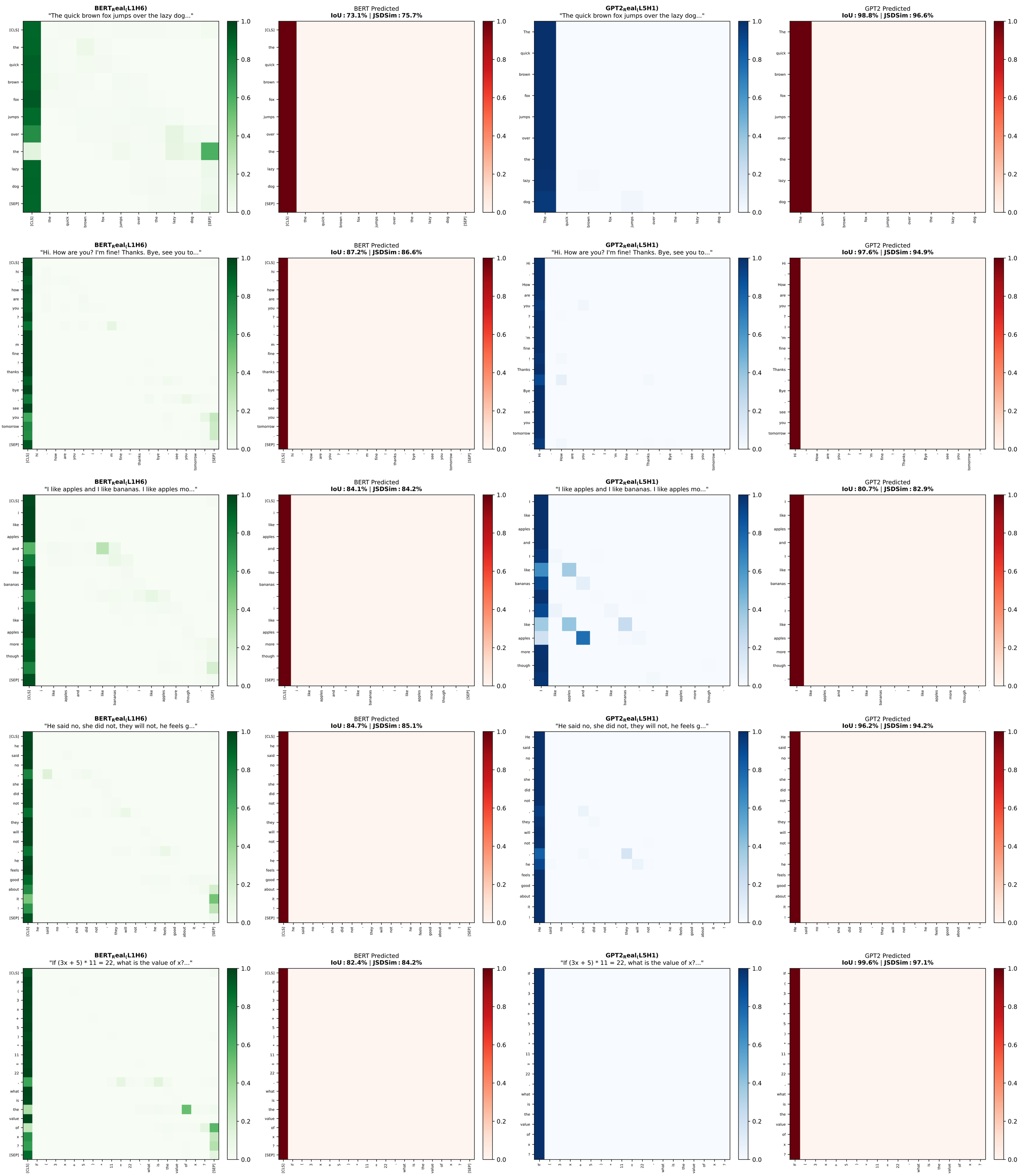


GPT2 Predicted
IoU : 94.7% | JSDSim : 94.0%



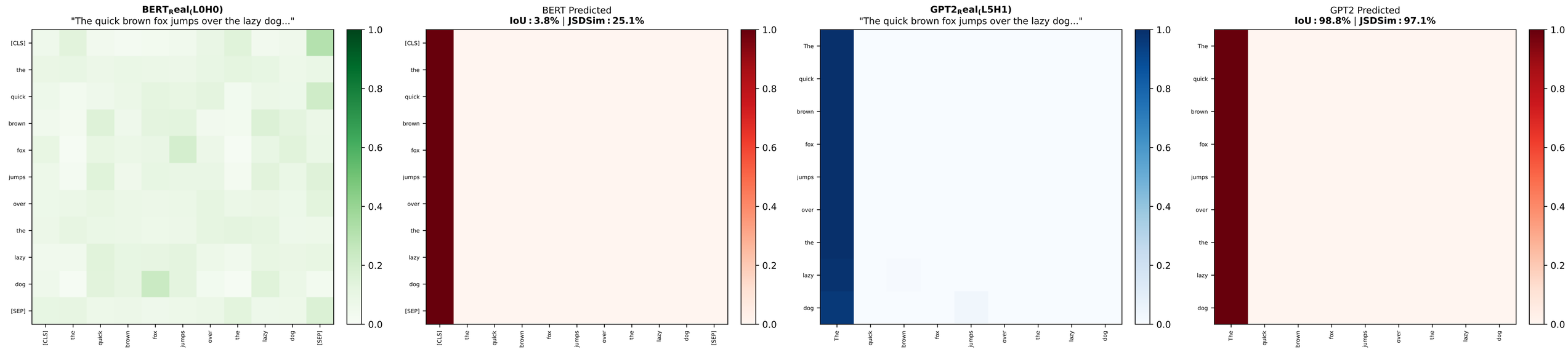
Pattern: initial_token_anchoring

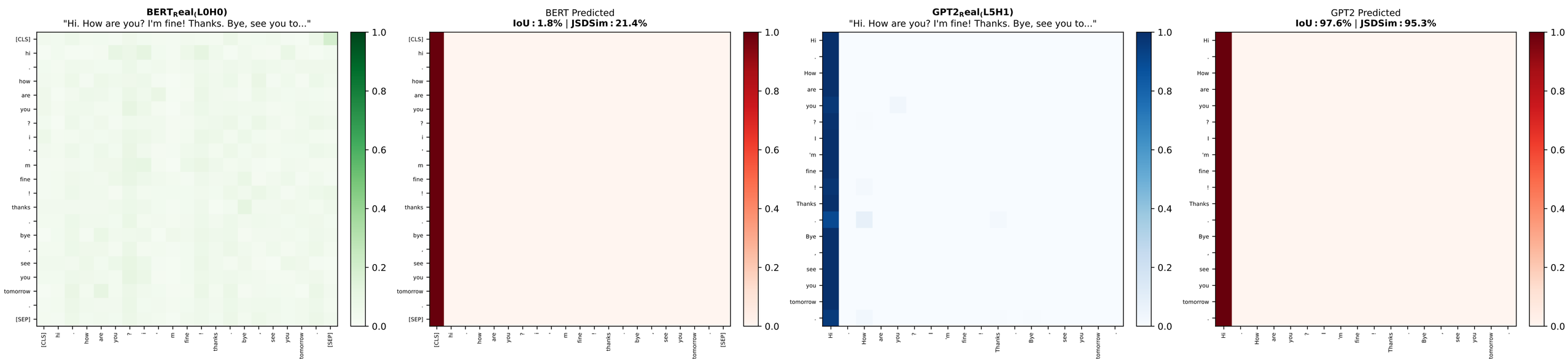
```
def initial_token_anchoring(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    # Set strong attention from all tokens to the initial token
    for i in range(1, len_seq-1):
        out[i, 0] = 1
    # Ensure the [CLS] and [SEP] have self-attention
    out[0, 0] = 1
    out[-1, 0] = 1
    out += 1e-4  # Regularization to prevent complete sparsity
    out /= out.sum(axis=1, keepdims=True)  # Normalize attention
    return "Initial Token Anchoring", out
```

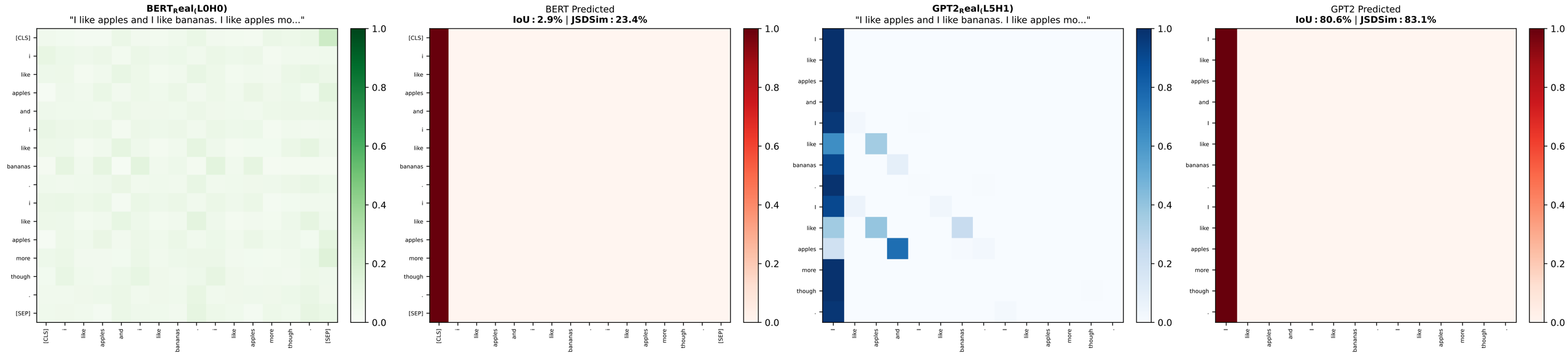


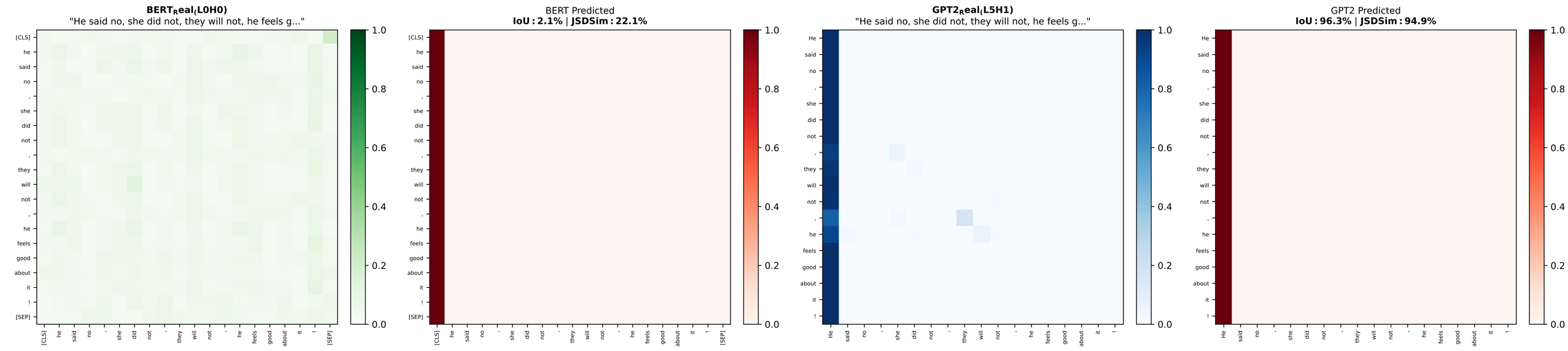
Pattern: initial_token_attachment

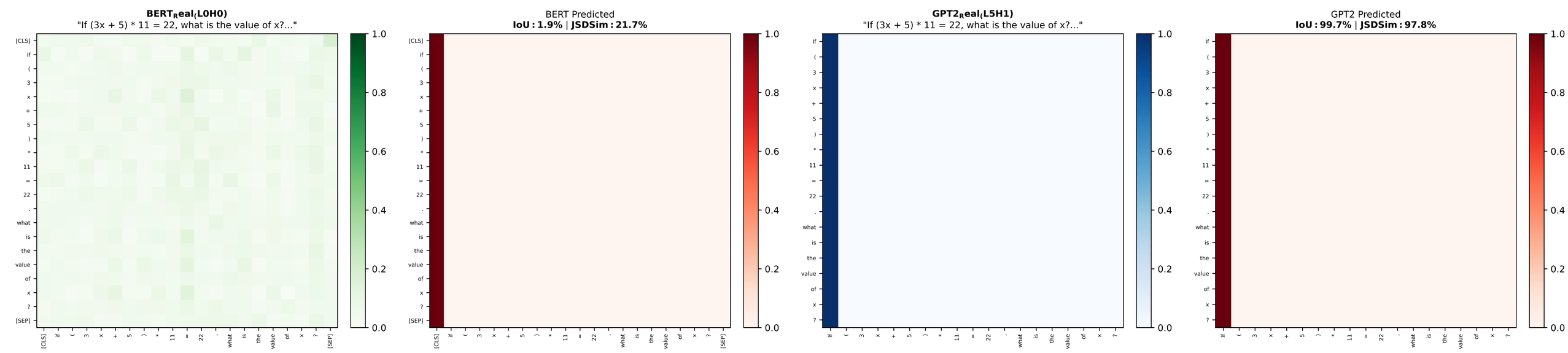
```
def initial_token_attachment(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Attach all tokens strongly to the first token
    for i in range(1, len_seq):
        out[i, 0] = 1

    # Add CLS and EOS attention
    out[0, 0] = 1
    out[-1, 0] = 1

    # Normalize the attention scores across each row
    out = out / out.sum(axis=1, keepdims=True)
    return "Initial Token Attachment", out
```





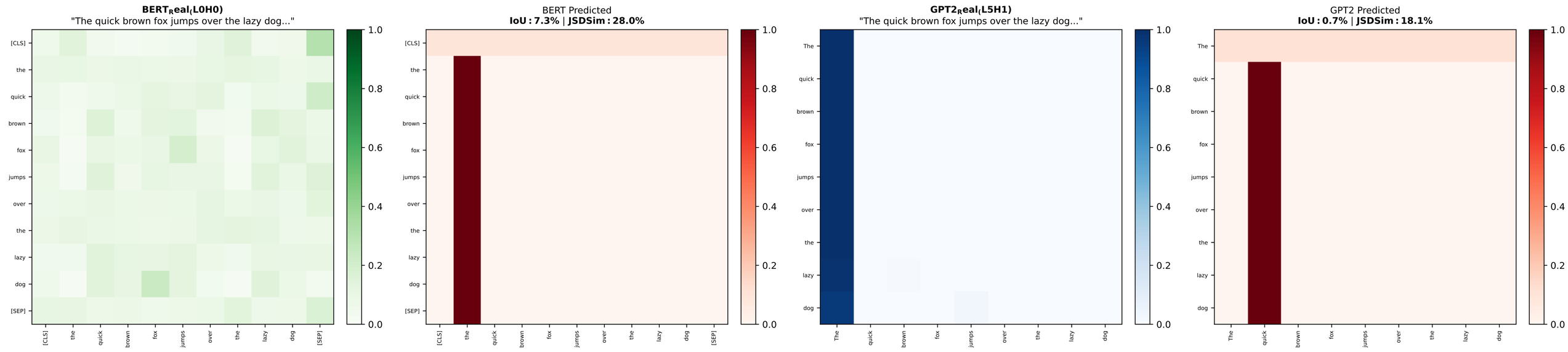


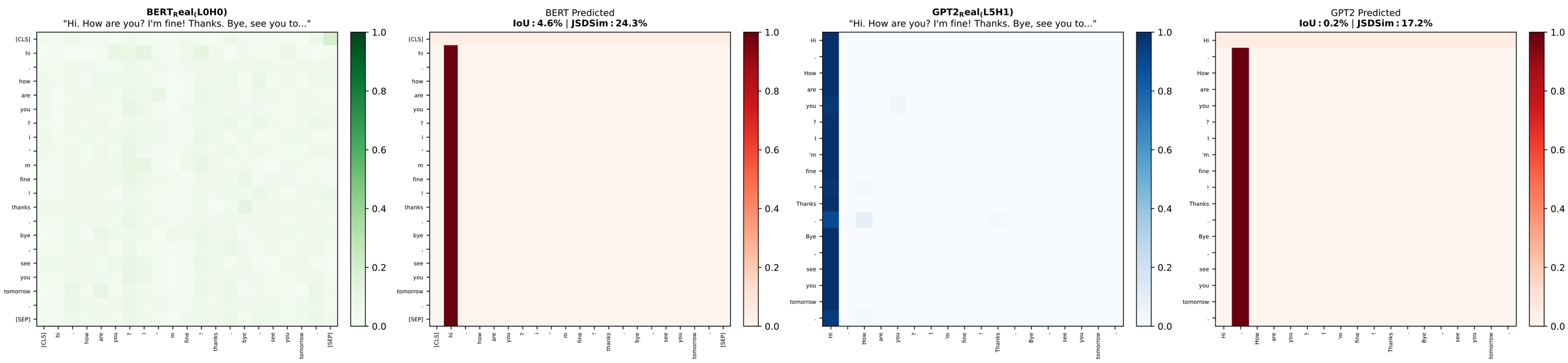


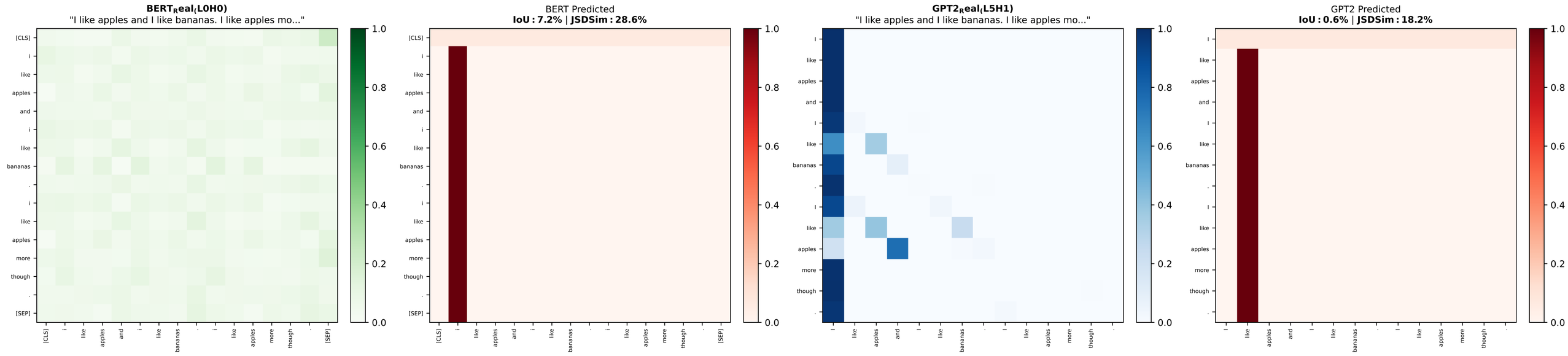


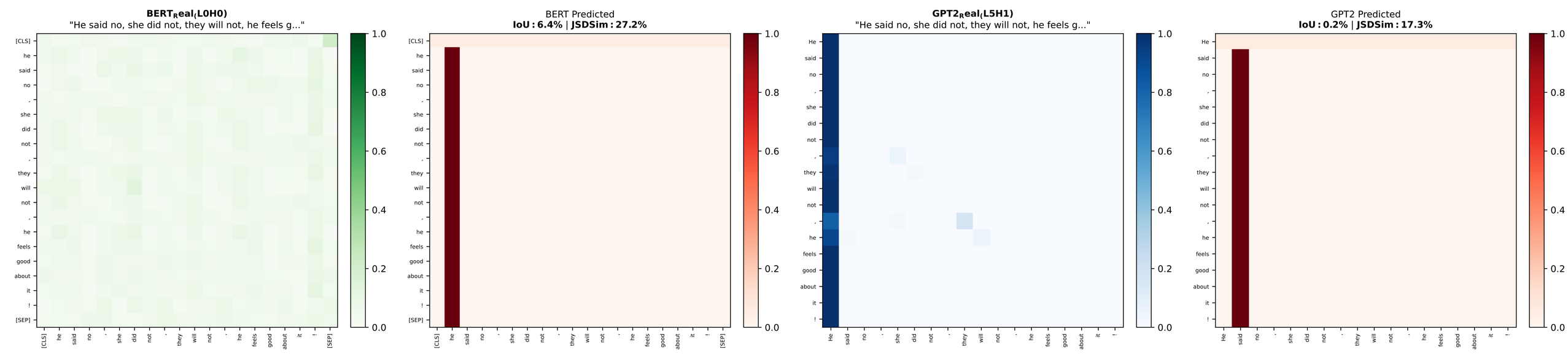
Pattern: initial_token_attention

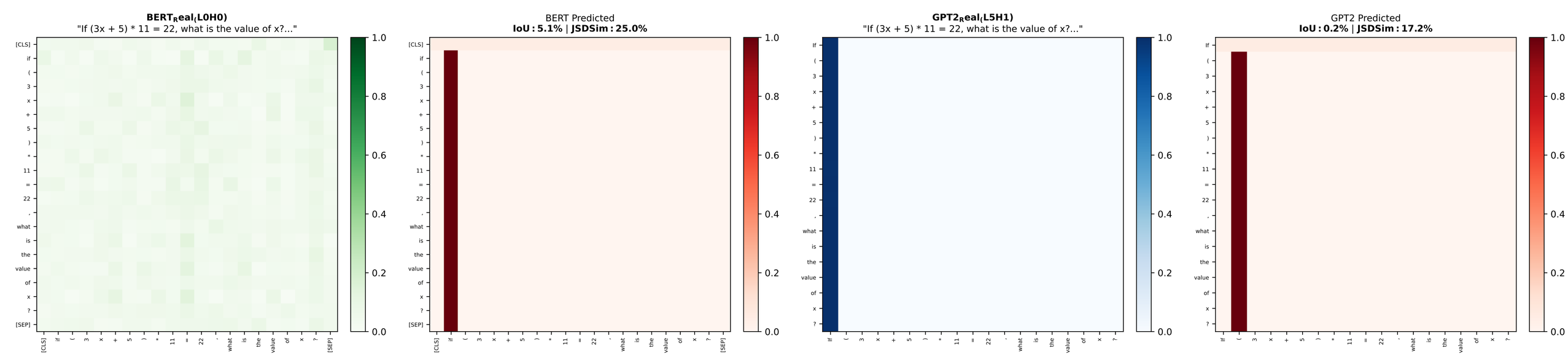
```
def initial_token_attention(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # The initial token (after CLS) seems to receive the majority of attention.
    for i in range(1, len_seq):
        out[i, 1] = 1.0

    # Normalize each row to ensure the sum is 1 (standard practice in attention heads)
    out += 1e-4  # Avoid division by zero
    out = out / out.sum(axis=1, keepdims=True)

    return "Initial Token Attention Pattern", out
```





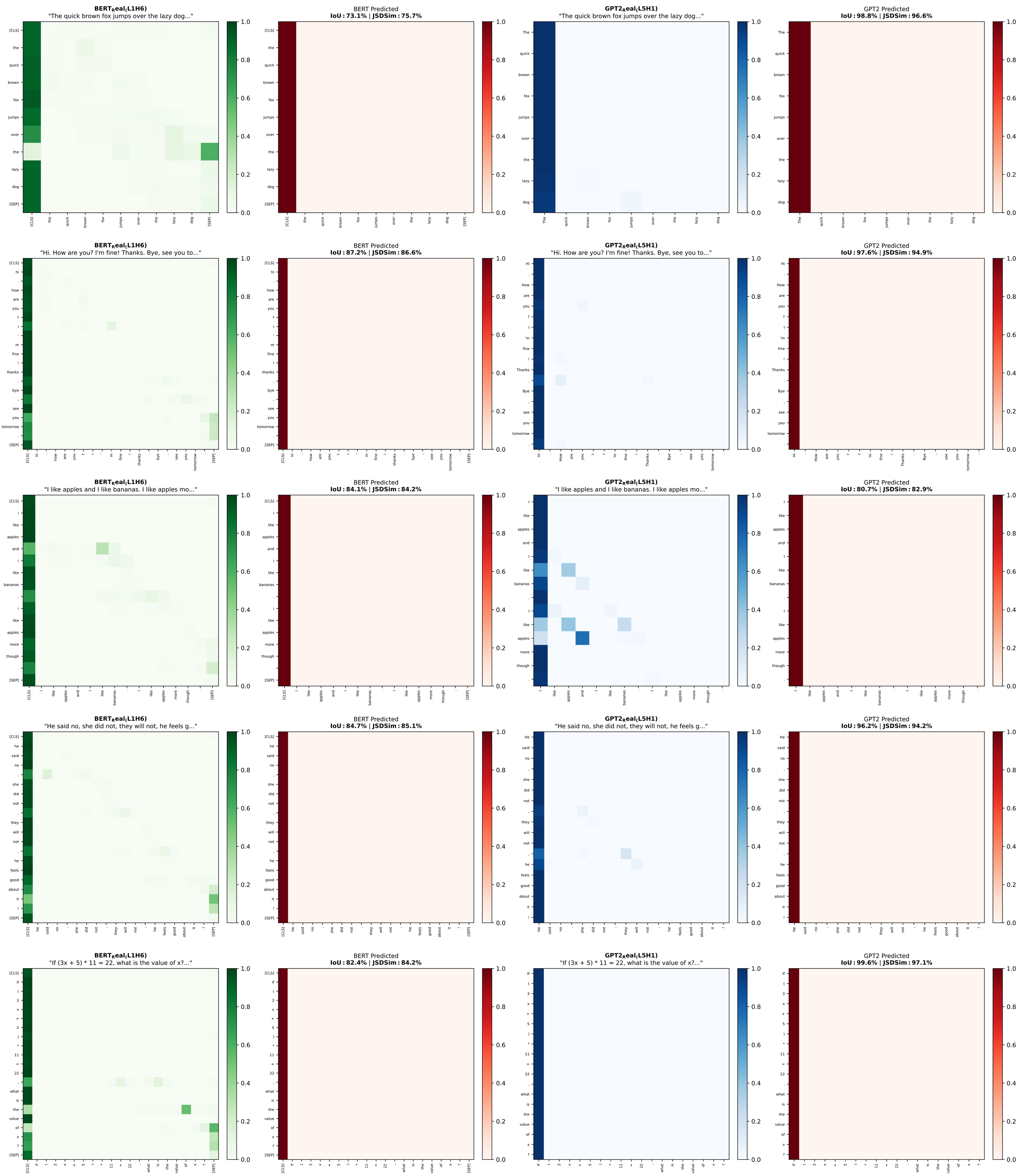






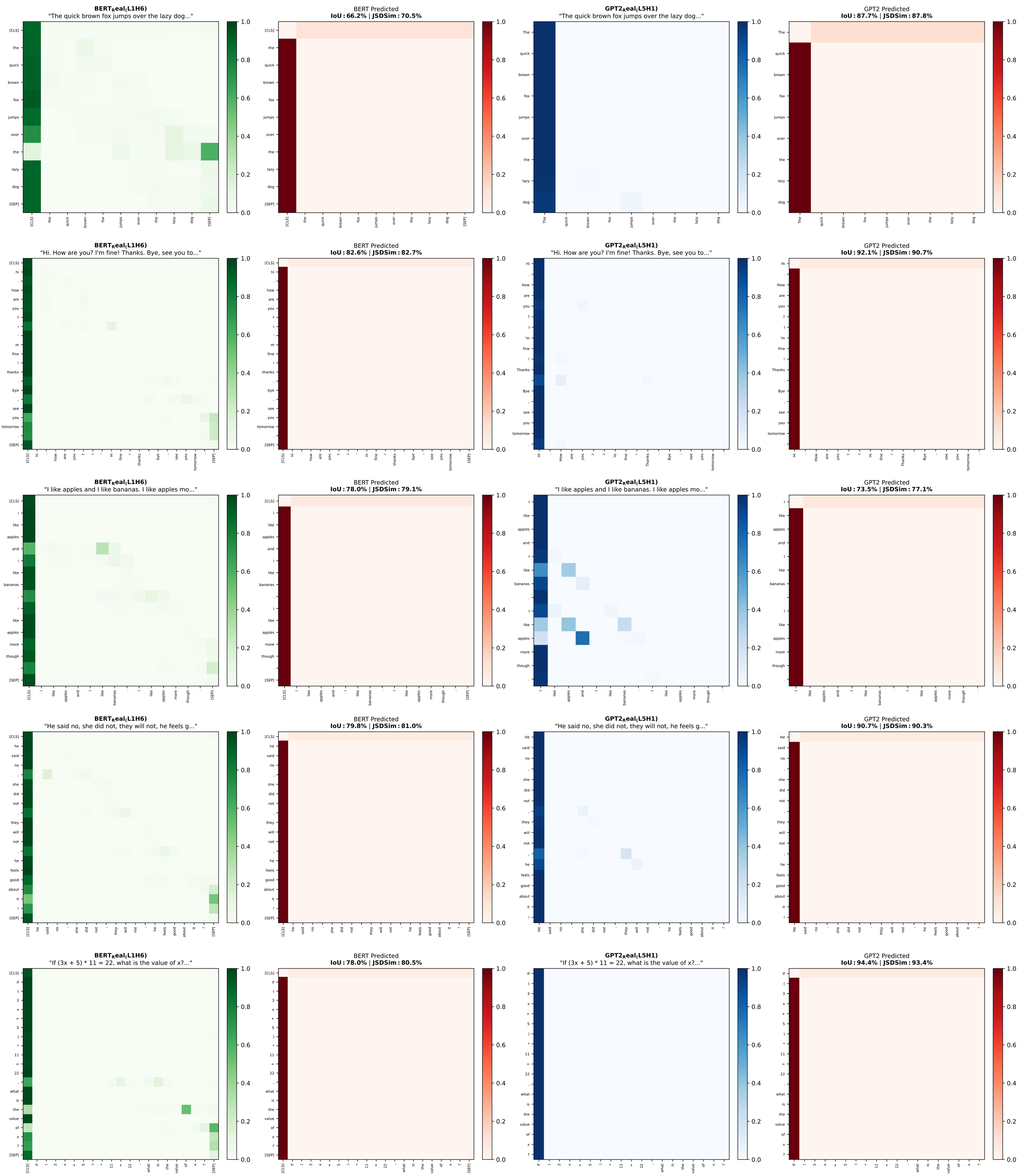
Pattern: initial_token_centralization

```
def initial_token_centralization(sentence: str, tokenizer: PreTrainedTokenizerBase):
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # Assign maximum attention to the first token for simplicity,
# adjusted for GPT-2 tokenizer alignment (consider CLS, EOS center tokens or padding as needed)
    for i in range(len_seq):
out[i, 0] = 1  # All tokens pay strong attention to the first token, similar to an initial centralization

    # Normalize the output matrix row-wise
    out += 1e-4
    out = out / out.sum(axis=1, keepdims=True)

    return "Initial Token Centralization", out
```



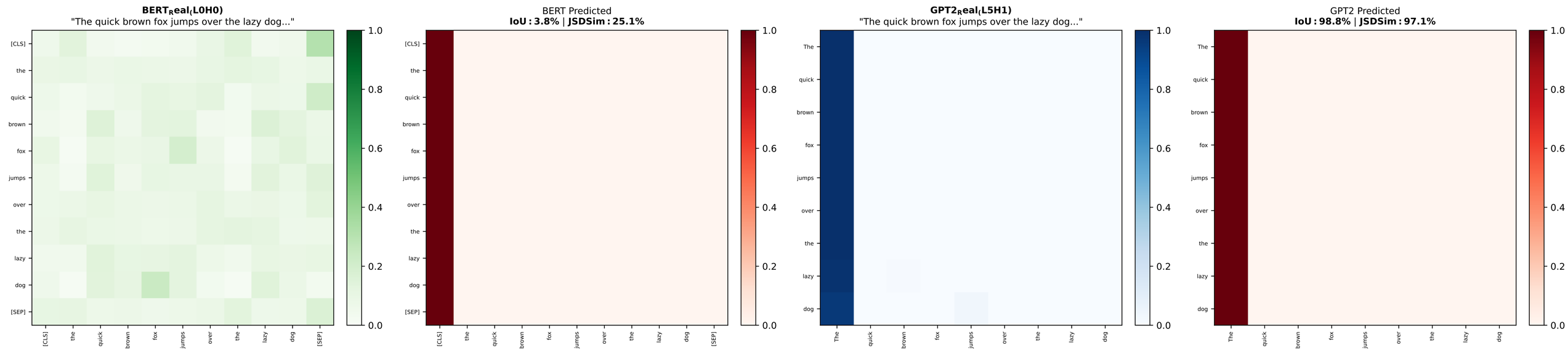
Pattern: initial_token_dominance

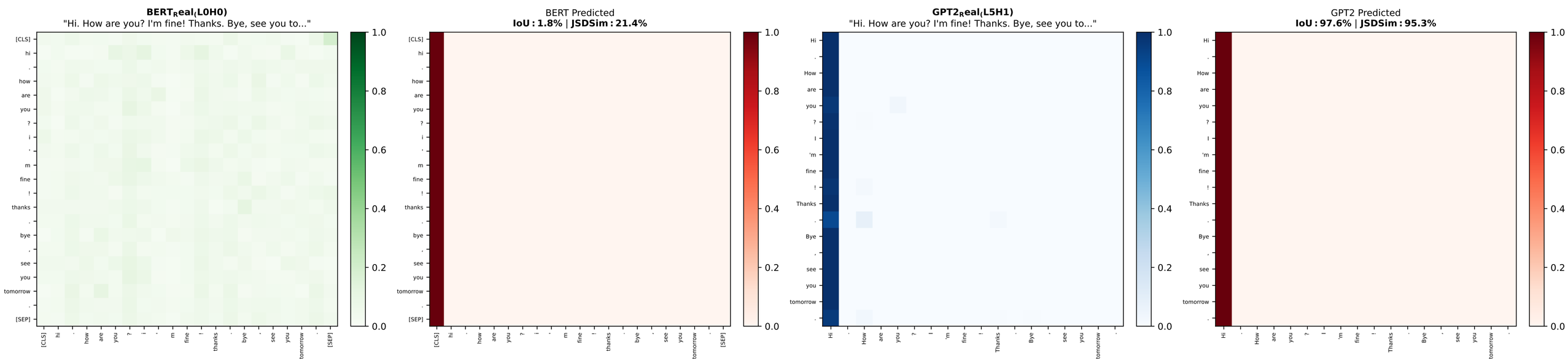
```
def initial_token_dominance(sentence: str, tokenizer: PreTrainedTokenizerBase):
                    toks = tokenizer([sentence], return_tensors="pt")
                             len_seq = len(toks.input_ids[0])
                           out = np.zeros((len_seq, len_seq))
                # Most significant attention comes from the first token
                   out[0, :] = 1/(len_seq - 1) if len_seq > 1 else 0
      # Define non-zero attention over all tokens that are not the CLS token
                                  if len_seq > 1:
                                     out[0, 0] = 0
                           # Apply normalization over columns
                                  if len_seq > 1:
                                    out[:, 0] = 1e-4
                      out = out / out.sum(axis=1, keepdims=True)
                        return 'Initial Token Dominance', out
```

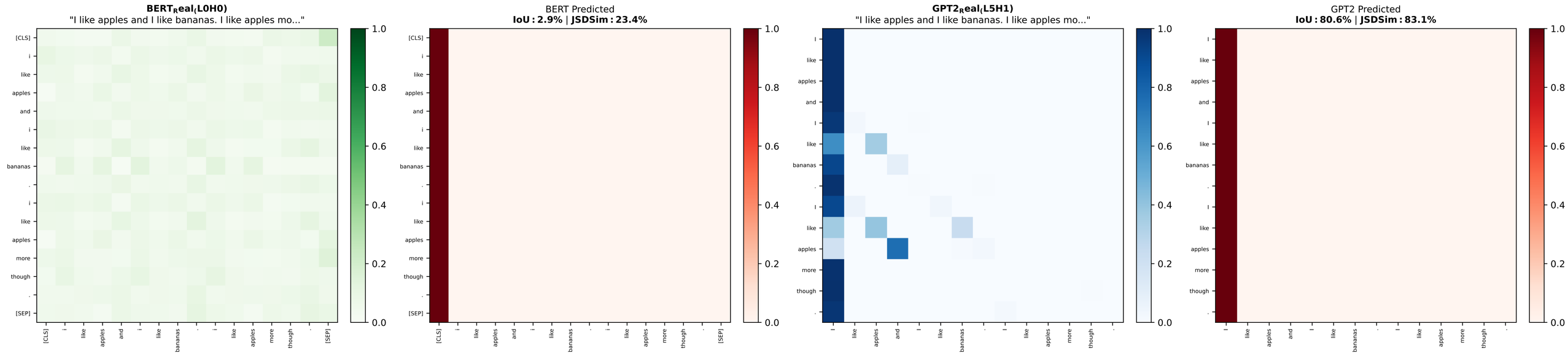


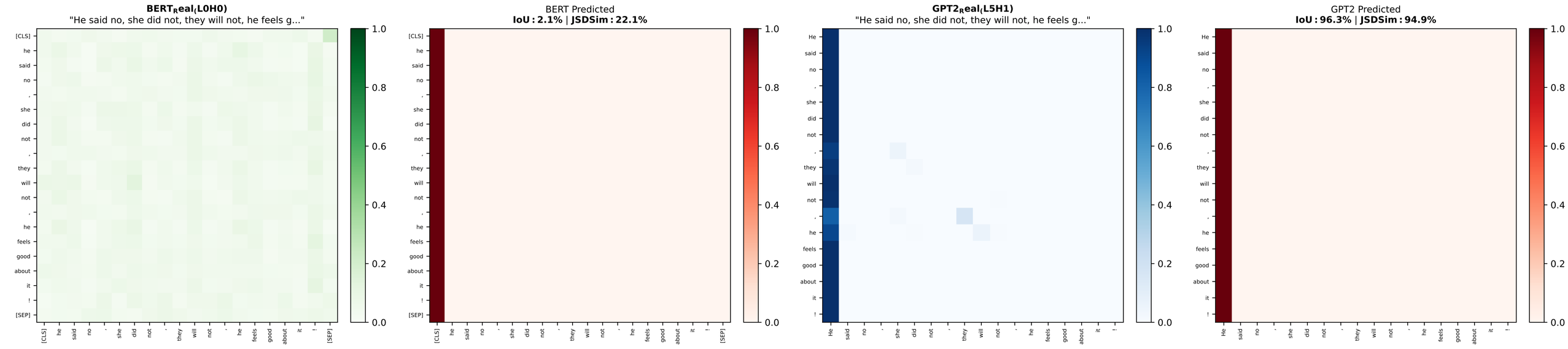
Pattern: initial_token_emphasis

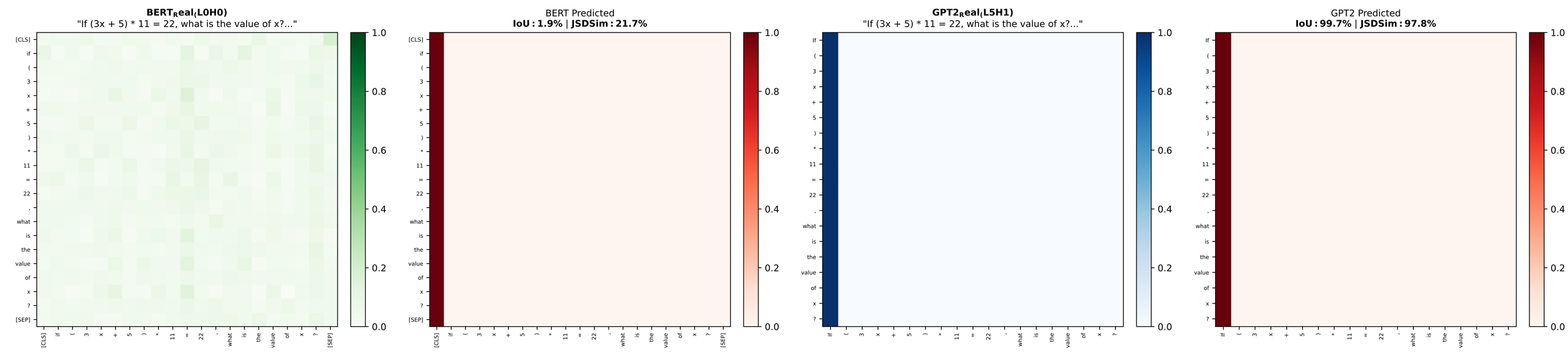
```
def initial_token_emphasis(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple[str, np.ndarray]:
                                toks = tokenizer([sentence], return_tensors="pt")
                                         len_seq = len(toks.input_ids[0])
                                        out = np.zeros((len_seq, len_seq))
             # Emphasize the initial token by assigning high attention to subsequent tokens
                              # Normalize by sequence length adaptively per token
                  for i in range(1, len_seq):  # start from the first meaningful token
                                             out[i, 0] = 1.0

                          # Add self attention for the [CLS] token at the start
                                          out[0, 0] = 1.0

                         # Normalize the attention distribution across each row
                               out /= np.sum(out, axis=1, keepdims=True)
                                 return "Initial Token Emphasis", out
```





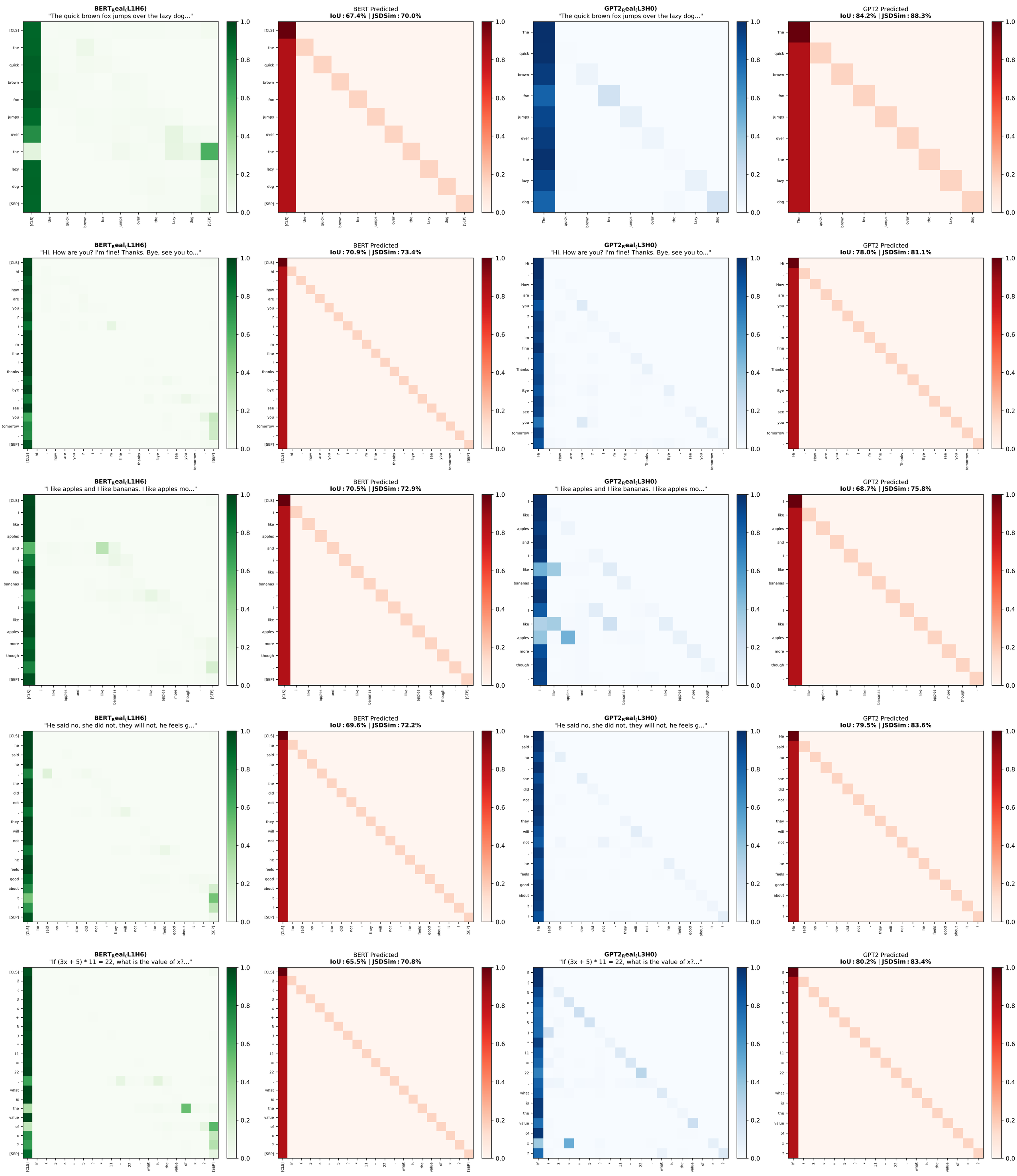


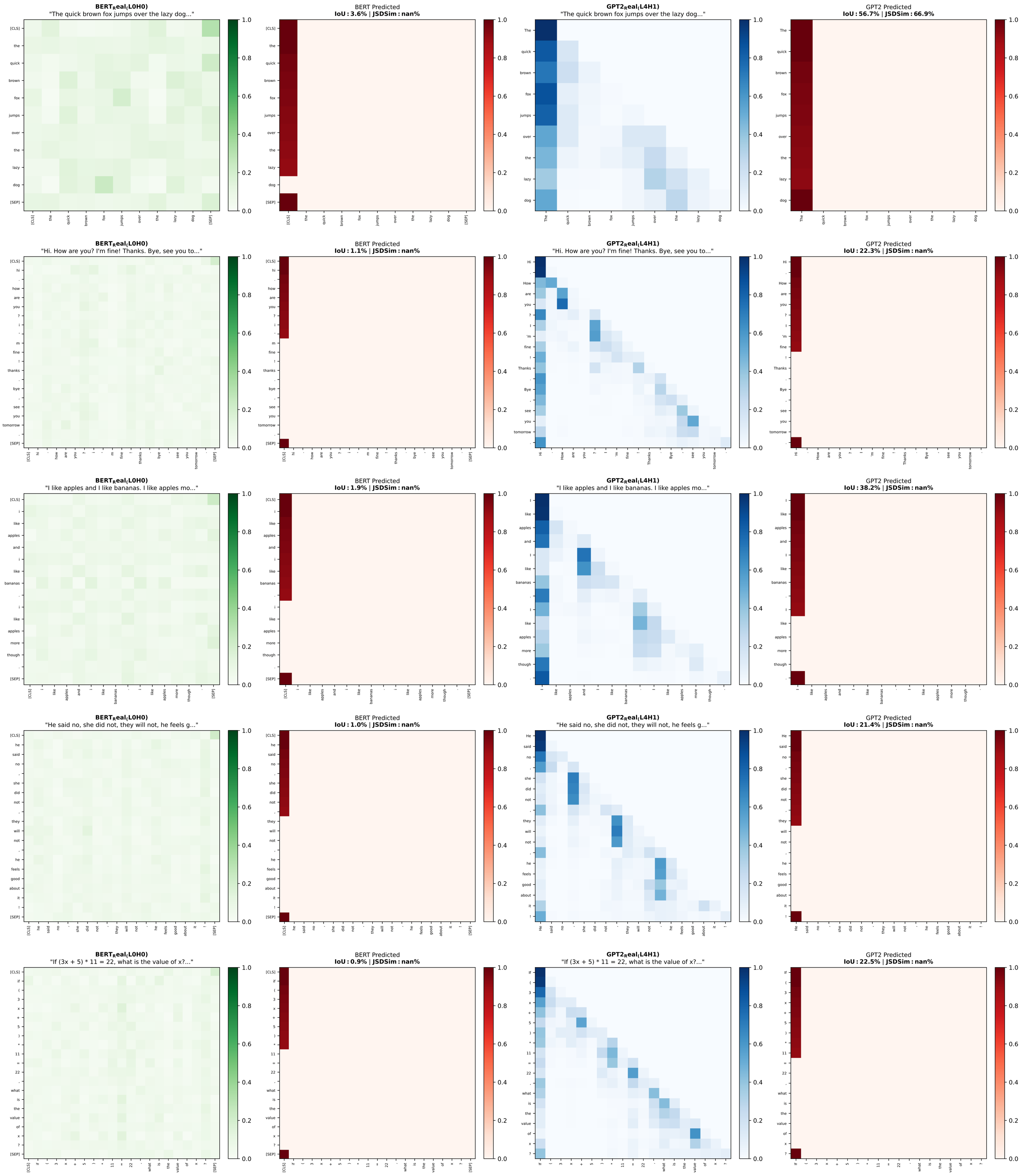




Pattern: initial_token_reference_attention

```
def initial_token_reference_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> Tuple[str, np.ndarray]:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))

    # High self-attention for the first token
    out[0, 0] = 1.0
    for i in range(1, len_seq):
        # Moderate to low attention from each token to the initial token
        out[i, 0] = 0.5
        # Higher attention from the initial token to itself
        out[i, i] = 0.1

    # Normalize attention weights per row
    out = out / out.sum(axis=1, keepdims=True)

    return "Initial Token Reference Attention Pattern", out
```



Pattern: initial_word_attention

```
def initial_word_attention(sentence: str, tokenizer: PreTrainedTokenizerBase) -> tuple:
    toks = tokenizer([sentence], return_tensors="pt")
    len_seq = len(toks.input_ids[0])
    out = np.zeros((len_seq, len_seq))
    attention_values = [100, 97, 96, 95, 94, 93, 92, 91]
    for i in range(1, min(len_seq-1, len(attention_values)+1)):
        out[i, 0] = attention_values[i-1] / 100
    out[0, 0] = 1
    out[-1, 0] = 1
    return "Initial Word Attention", out
```